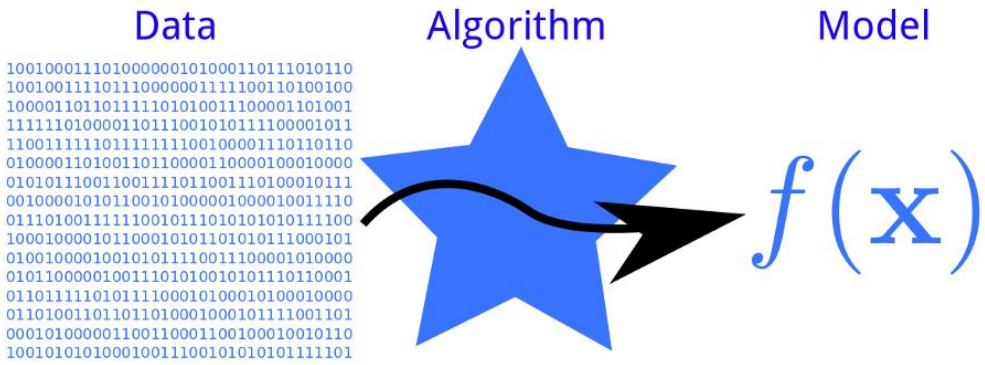
经典机器学习



机器学习

1.1 机器学习概述

1.2 机器学习任务

1.3 机器学习评价

1.4 决策树模型及应用

1.1 机器学习概述

- 什么是学习？AI角度

西蒙（Simon）对学习的定义：“如果一个系统能够通过执行某种过程而改进它的性能，这就是学习”。

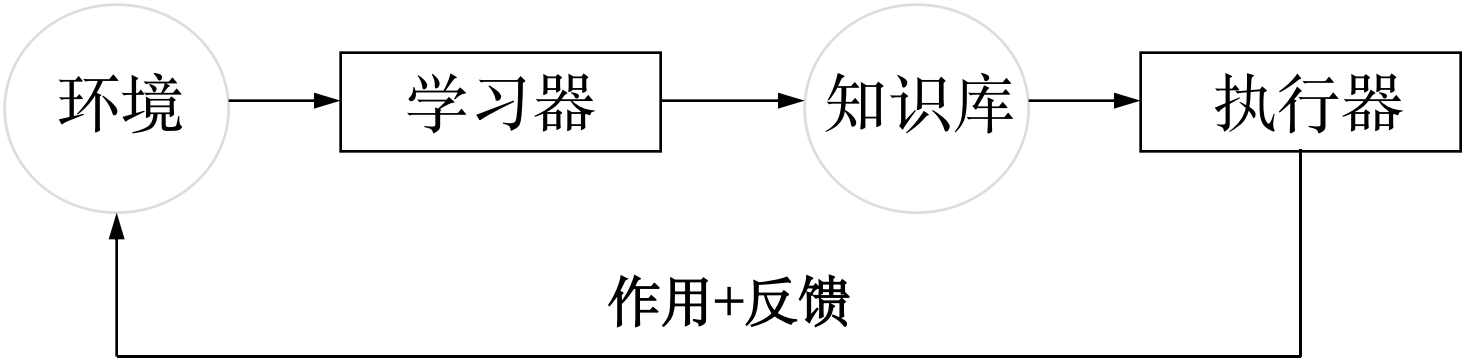
学习的三个的要点：

- ✓ 学习是一个过程
- ✓ 学习是对一个系统而言
- ✓ 通过学习优化系统性能



1.1 机器学习概述

- 学习系统的一般结构



✓从环境获得信息，学习器负责加工信息形成知识，以提升系统执行任务的性能；

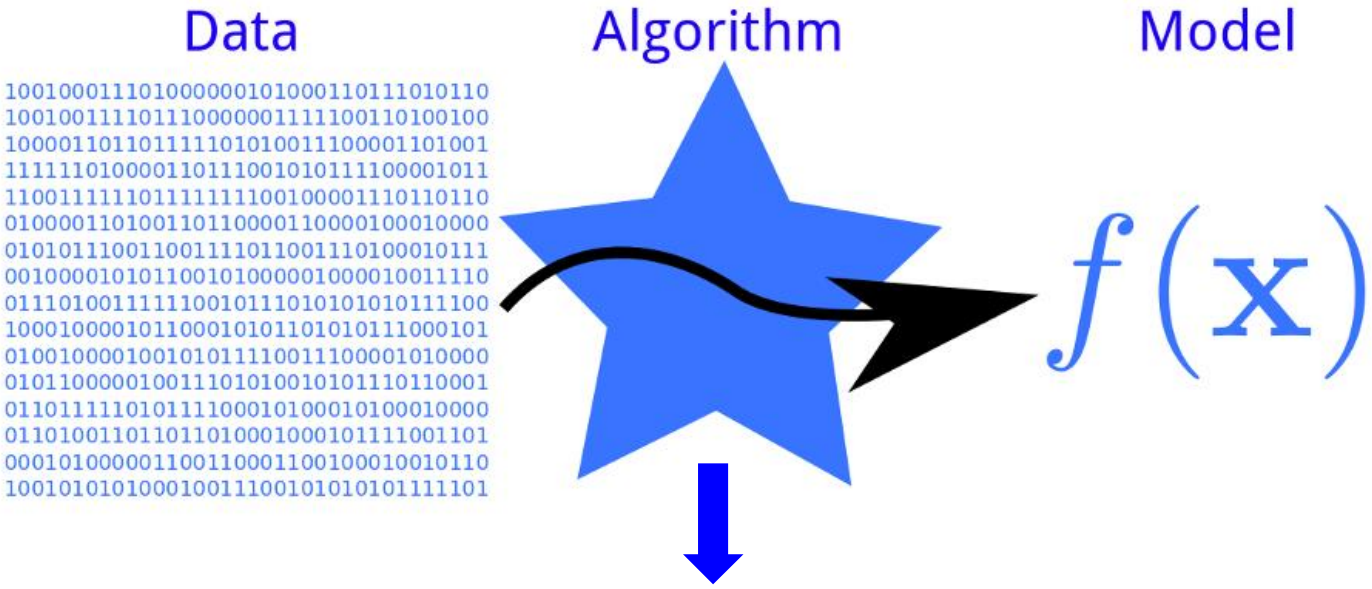
✓执行器根据获得的知识指导任务的完成，同时把获得的信息进一步反馈给环境；

✓环境，知识库和执行器决定了学习系统要解决的问题。

1.1 机器学习概述

什么是机器学习（Machine Learning）？

- 机器学习是研究如何使用机器来模拟人类学习活动的一门学科；
- 让计算机从数据中进行自动学习，得到某种知识（或规律）；
- 机器学习在早期也被称为模式识别。

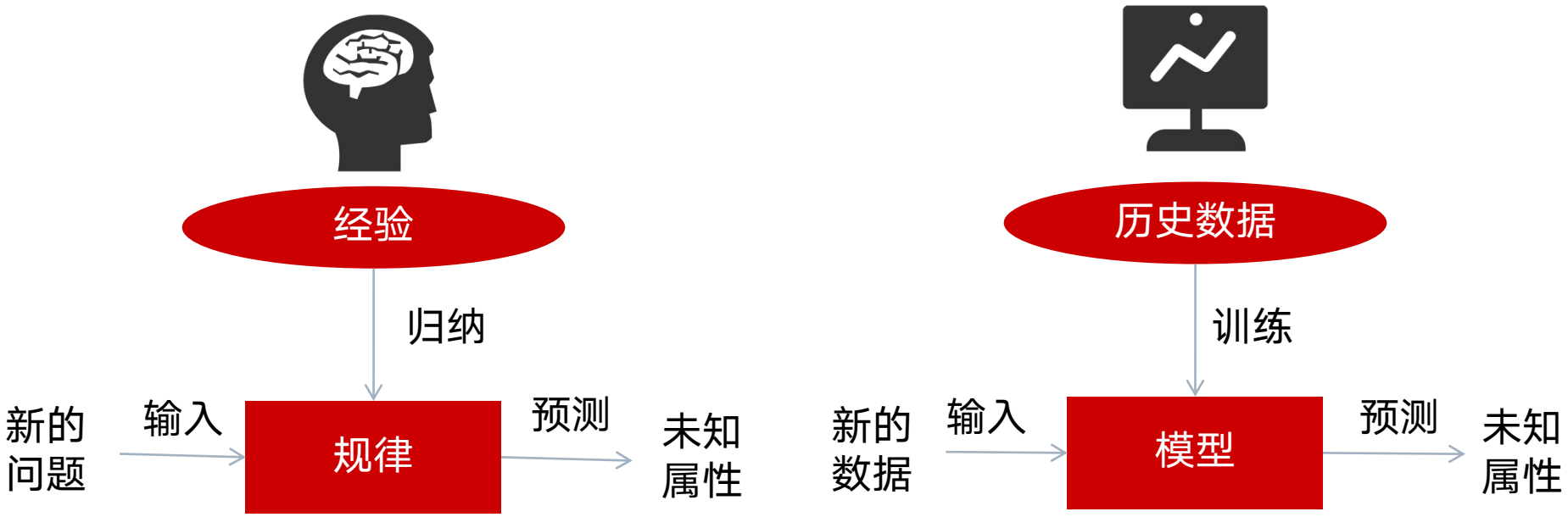


通过算法(Algorithm) 从大量历史数据(Data)中学习规律(Model)以实现对新数据的判别或预测。

1.1 机器学习定义

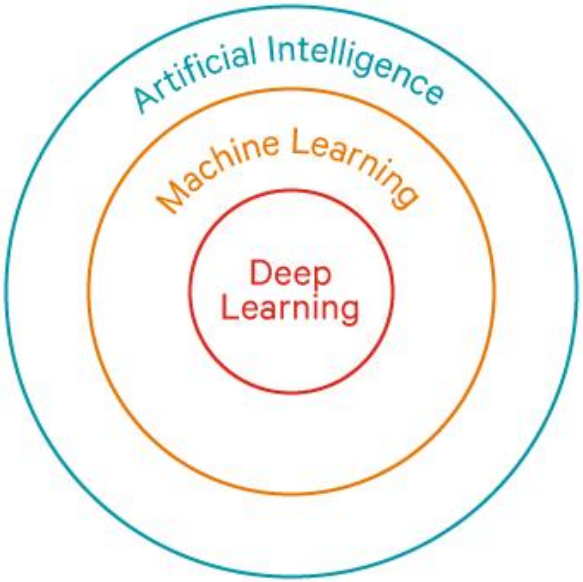
1. 机器学习（Machine Learning）是人工智能（AI）领域的一个子领域，旨在通过数据和经验，让计算机系统自动学习并从中做出预测或决策，而无需明确的编程指令。

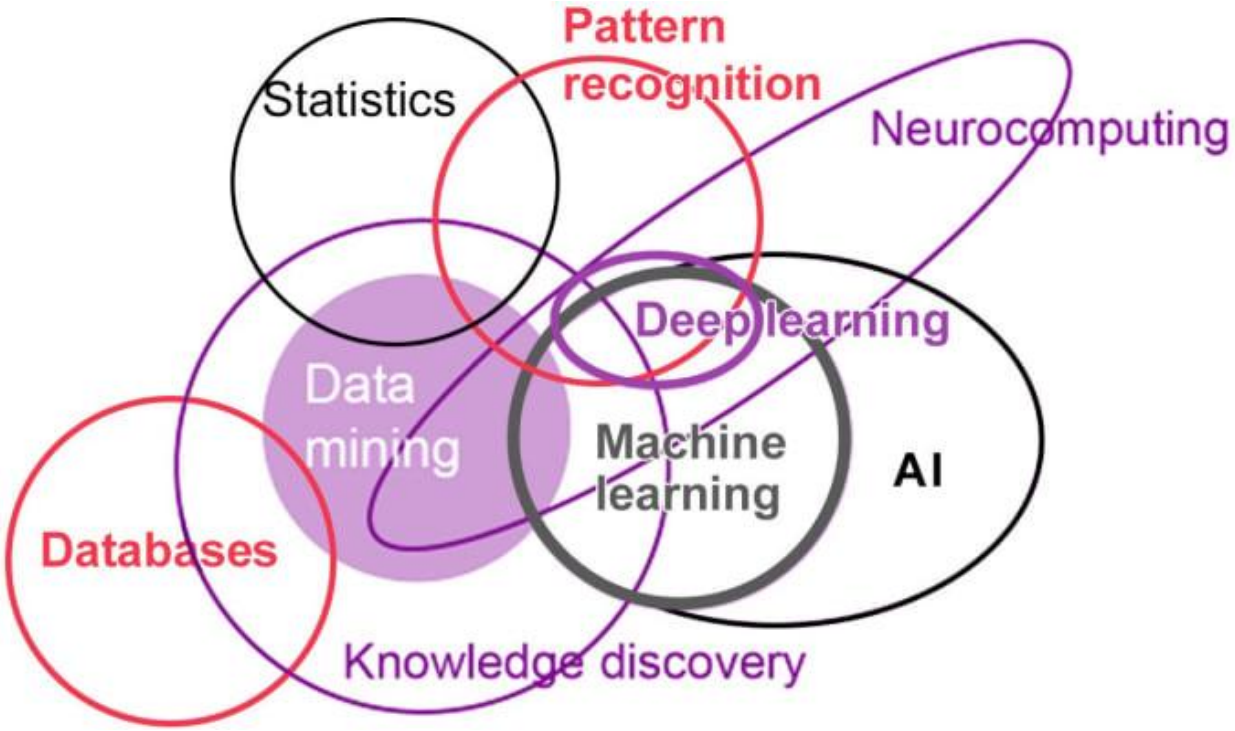
2. 机器学习算法通过分析大量的数据，识别其中的模式和规律，从而使模型能够在面对新的数据时进行自我优化。



1.1 机器学习概述

- 为什么机器学习很重要？





http://www.ciiip.com/news-11194-511.html

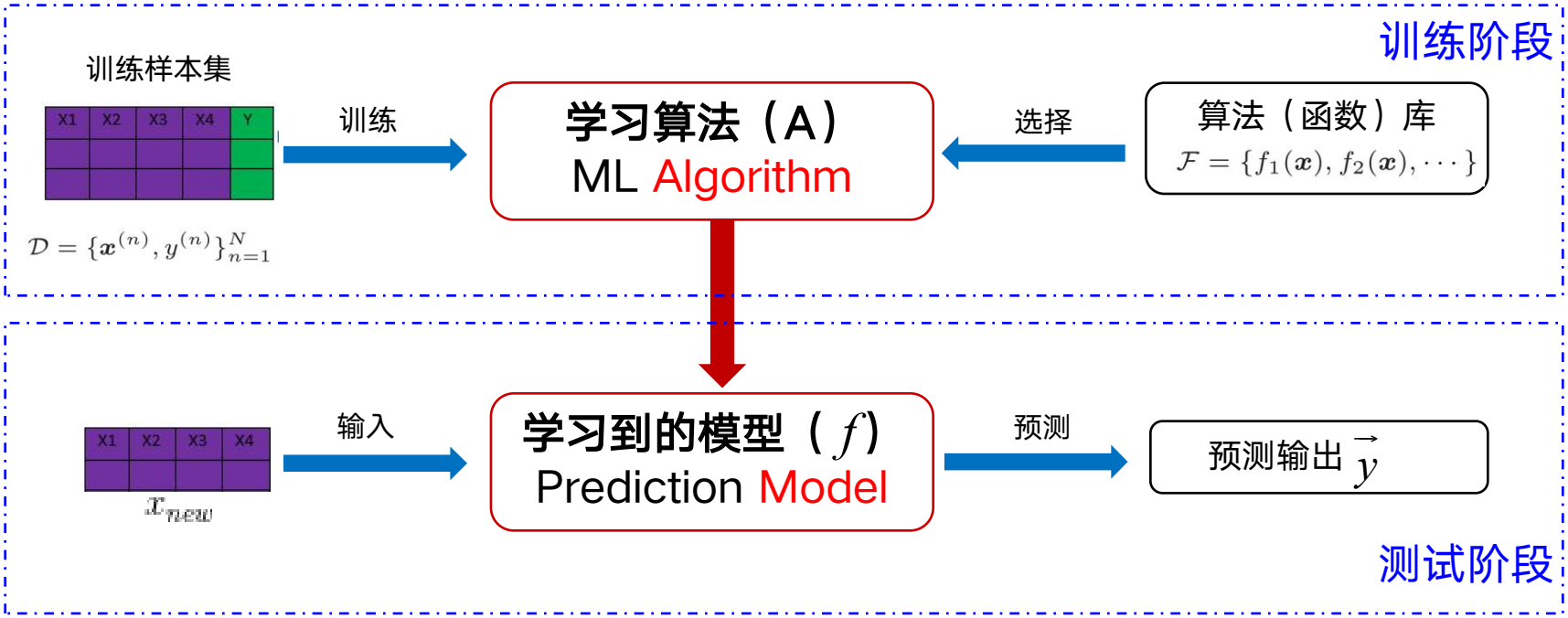
https://www.fiverr.com/bilalahmedd/machine-learning-data-science-tensor-flow-python

1.1 机器学习概述

■ 机器学习原理　问题表示

➢ 令X表示数据空间，Y表示目标空间。

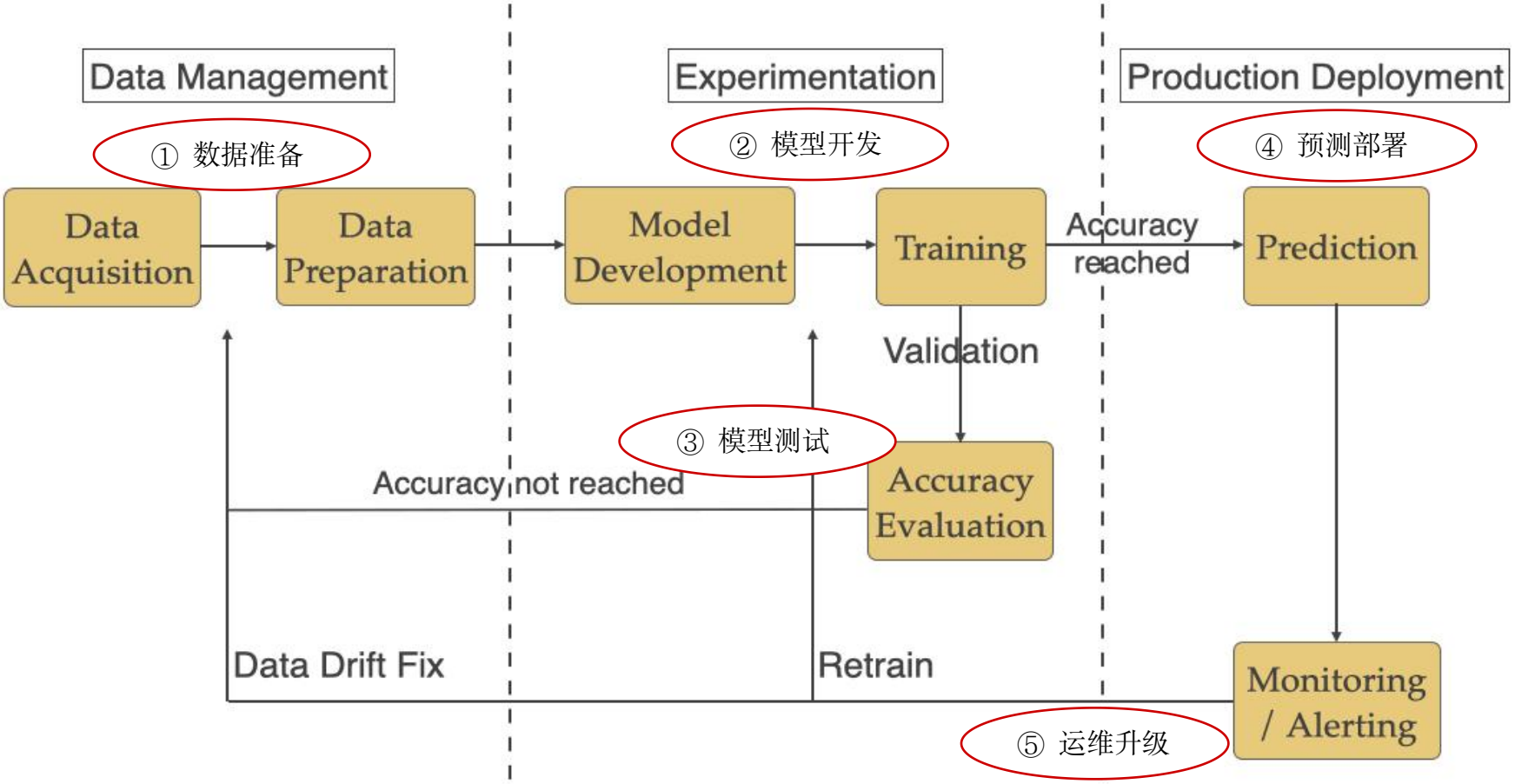
机器学习就是在已有数据基础上求得一个函数f(X):X→Y，实现从给定数据到目标空间的映射。



从大量的样本$\{(x_i, y_i)_{i=1}^{N}\}$中，寻找可以较好预测未见过x_{new}所对应的函数$f: x \to y$

1.1 机器学习概述

- 机器学习系统的一般过程



MLOps-机器学习运维

Machine Learning Operations

1.1 机器学习概述

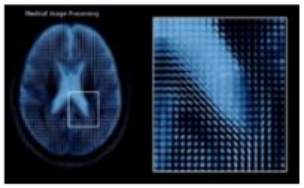
- 机器学习应用领域

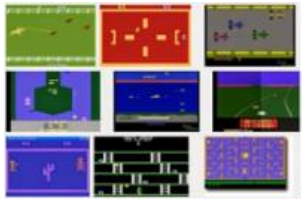

Self-driving


Surveillance detection



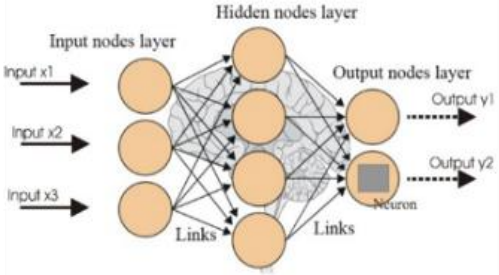
Translation


Medical diagnostics


Game


Personal assistant

Machine learning




Art

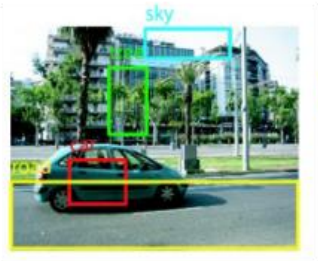


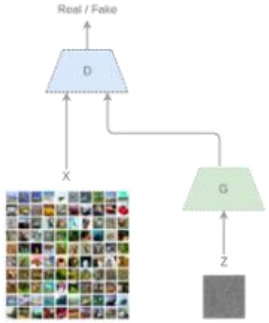
Image recognition



Speech recognition



Natural language



Generative model


Reinforcement learning

1.1 机器学习概述

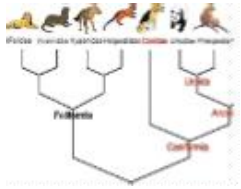
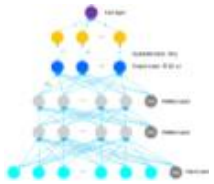
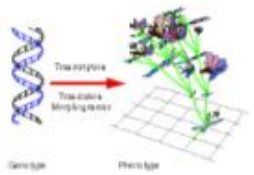
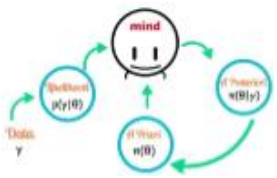
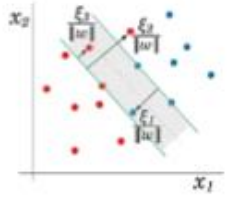
- 机器学习五大门派



1.1 机器学习概述

机器学习五大门派

机器学习五大门派

符号主义	连接主义	进化主义	贝叶斯派	类推派
				
基于符号表示、谓词逻辑的规则推理	基于神经元加权连接的数据自学习	基于生物遗传和DNA的进化学习	基于先验和假设的概率推理学习	基于相似性决策边界的类比学习
典型方法 规则/决策树 专家系统	典型方法 感知器 神经网络	典型方法 遗传算法 蚁群算法	典型方法 朴素贝叶斯 马尔科夫链	典型方法 支持向量机 K近邻算法

1.1 机器学习概述

- ML部分算法概览



机器学习

1.1 机器学习概述

1.2 机器学习任务

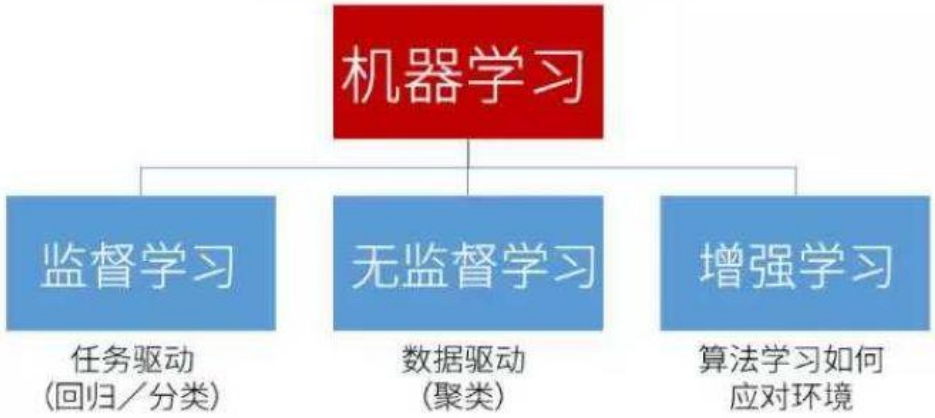
1.3 机器学习评价

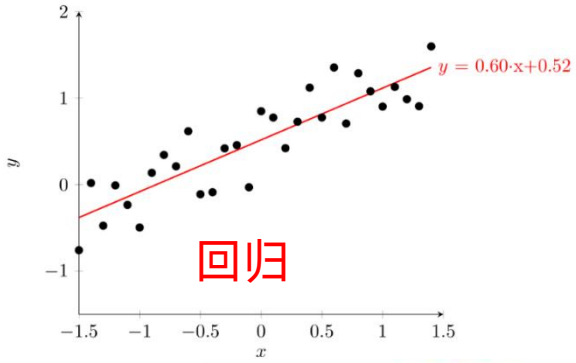
1.4 决策树模型及应用

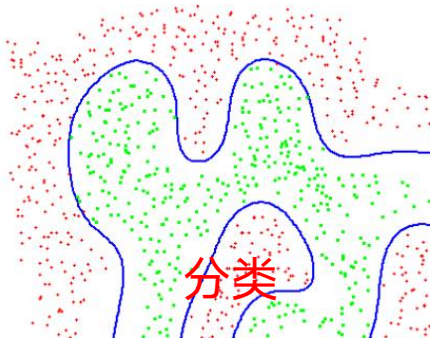
1.2 机器学习任务

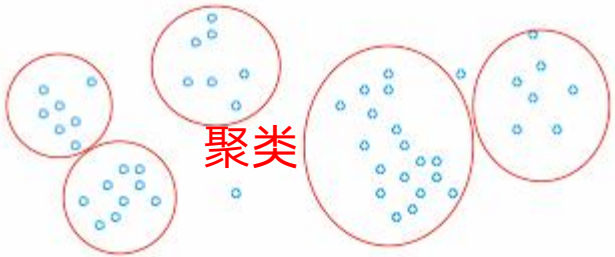
机器学习分类

- ✓ 按照有无指导来分：
  - 有监督学习（或有导师学习）
  - 非监督学习（或无导师学习）
  - 强化学习（或增强学习）
- ✓ 按推理策略来分：
  - 演绎学习
  - 归纳学习
  - 类比学习等
- ✓ 综合多因素的分类：
  - 人工神经网络学习
  - 深度强化学习
  - 进化学习等



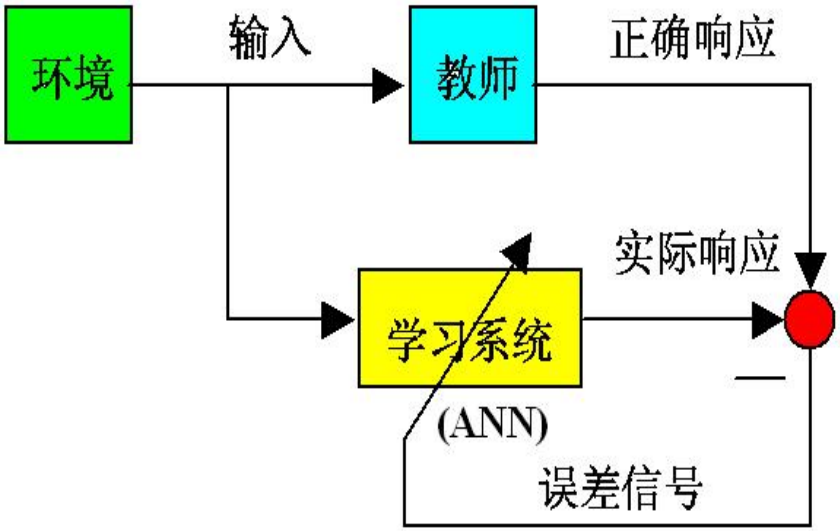


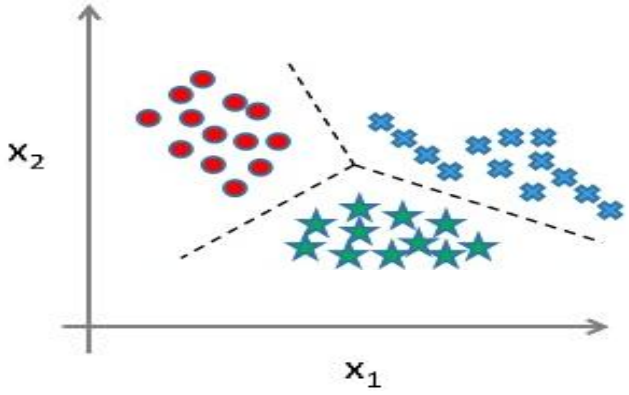


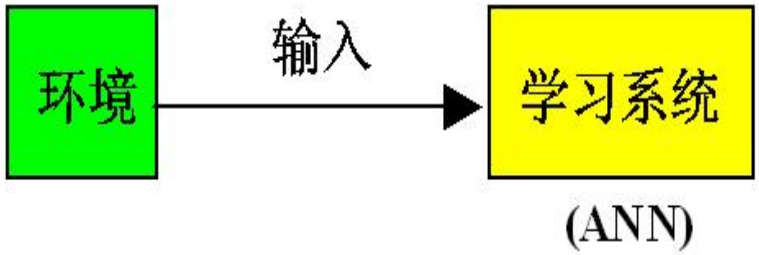


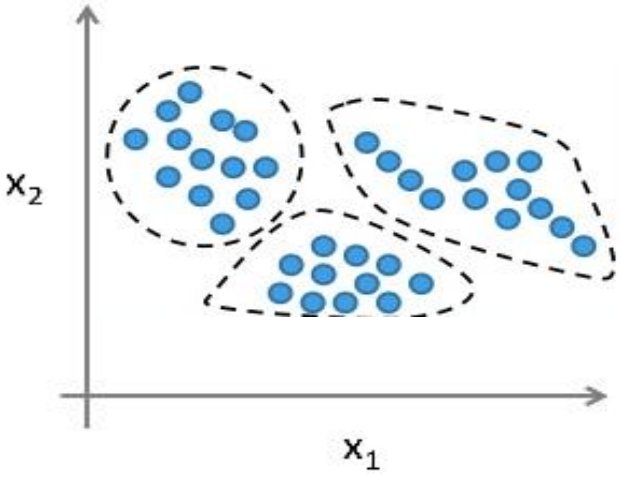
1.2 机器学习任务

- 机器学习分类：有监督学习 vs. 非监督学习



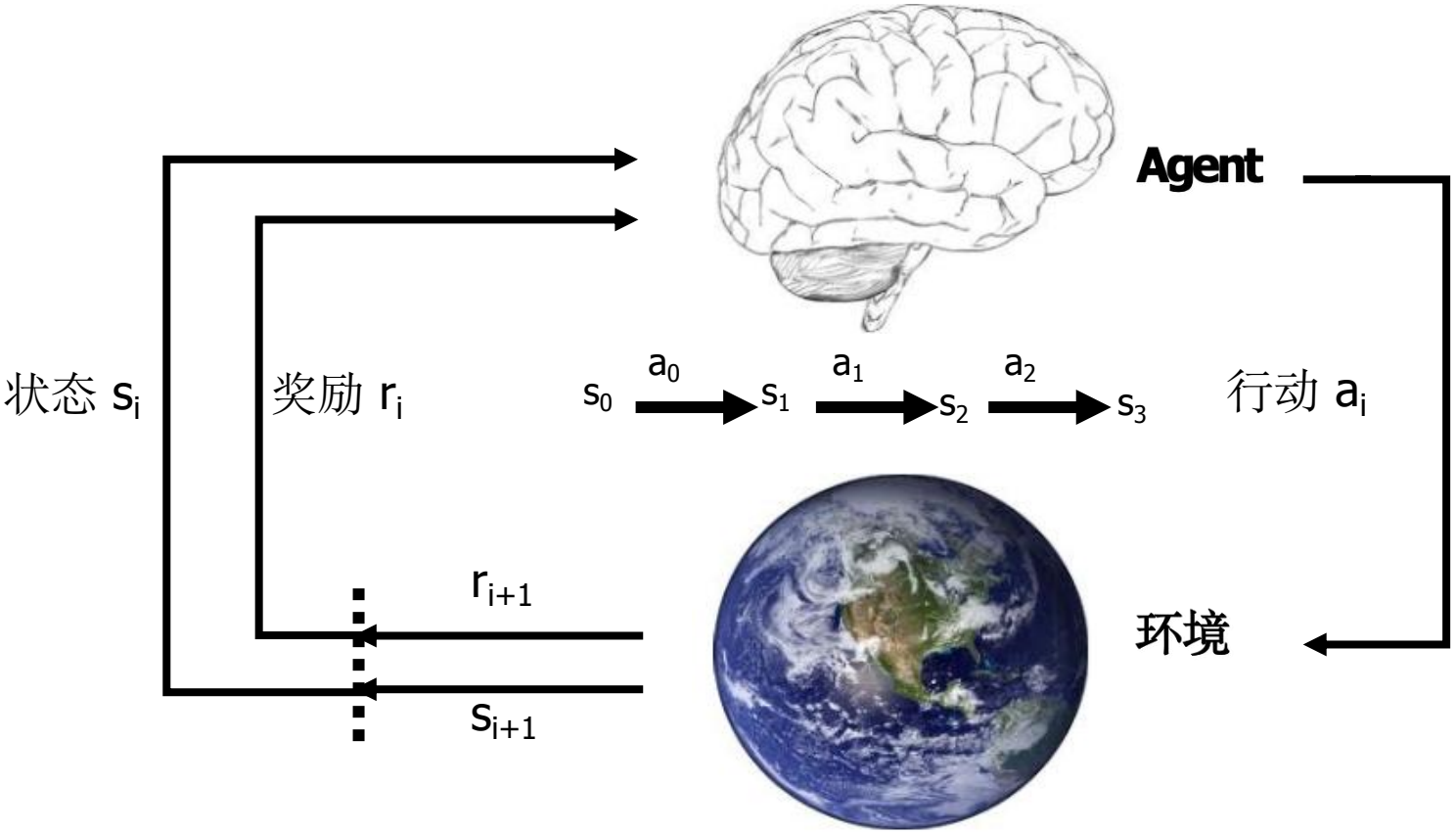






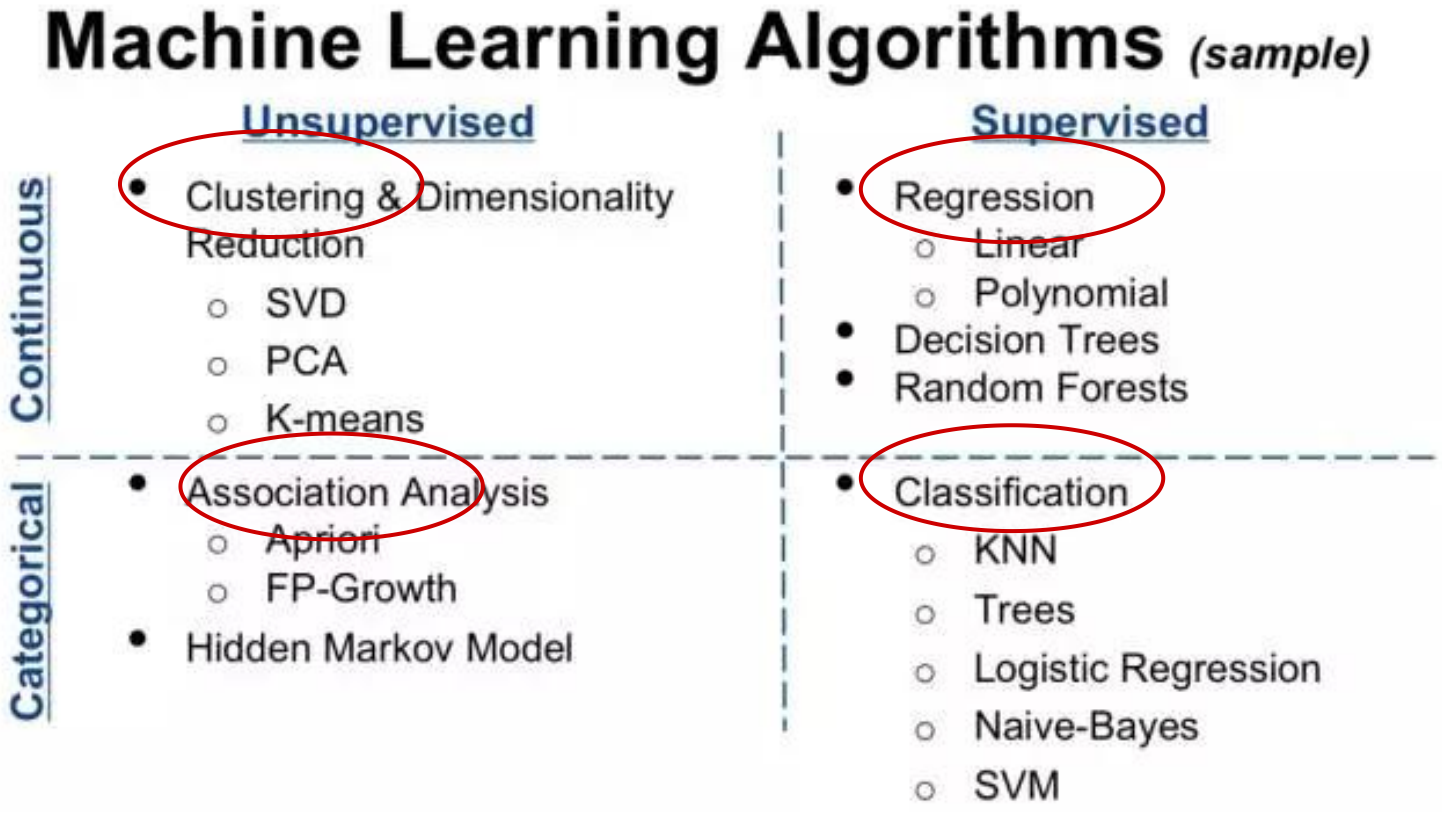
1.2 机器学习任务

- 机器学习分类：强化学习



1.2 机器学习任务

- **机器学习要解决的主要任务类型：**
  - ✓ 分类、聚类、回归（预测）、关联…



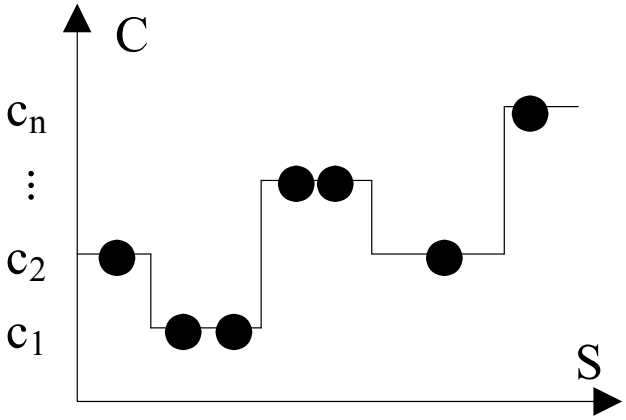
机器学习子集：深度学习（**CNN RNN LSTM GAN TRANSFORMER …**）

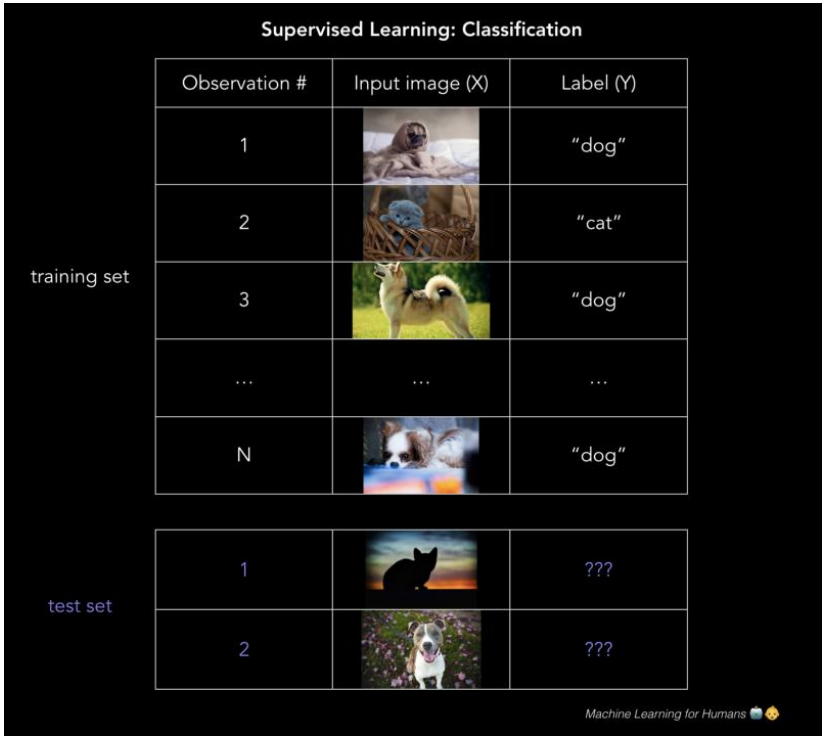
1.2 机器学习任务

- **分类问题：目标空间是已知有限离散值空间，**

  $Z=C=\{c_1,c_2,\ldots c_i,\ldots,c_n\}$ 待求函数就是分类模型

- ✓ 分类问题所用的训练数据是<D,C>
- ✓ 由于学习时目标类别已知，属于监督学习。
- ✓ 常用的方法：
  - 决策树、贝叶斯
  - 前馈神经网络BP
  - 支持向量机等



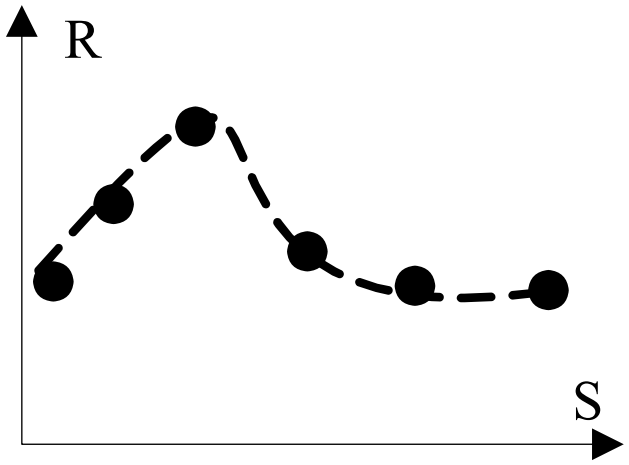


1.2 机器学习任务

- **回归问题：目标空间是连续值，**
  待求函数就是回归曲线，属于预测问题。

✓ 预测问题所用的训练数据是<D,R>，
- 事先已知（或选择了）曲线模型，需要学习的是模型中的参数。
- 例如已知多项式模型，但是要学习各项的系数。

✓ 常用的方法：
- 线性回归、非线性回归等。



Supervised Learning: Regression

	Observation #	Years of Higher Education (X)	Income (Y)
training set	1	4	$80,000
	2	5	$91,500
	3	0	$42,000
	4	2	$55,000
	…	…	…
	N	6	$100,000
test set	1	4	???
	2	6	???

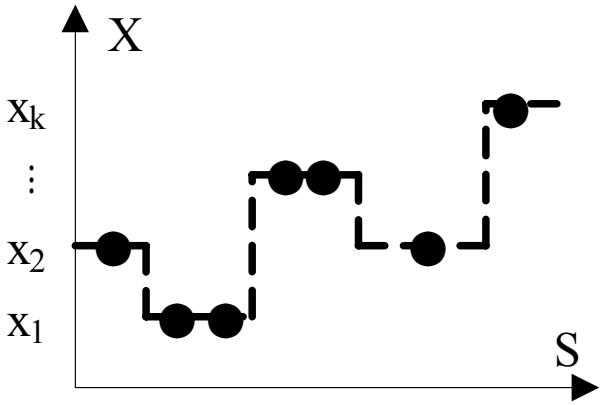
Machine Learning for Humans

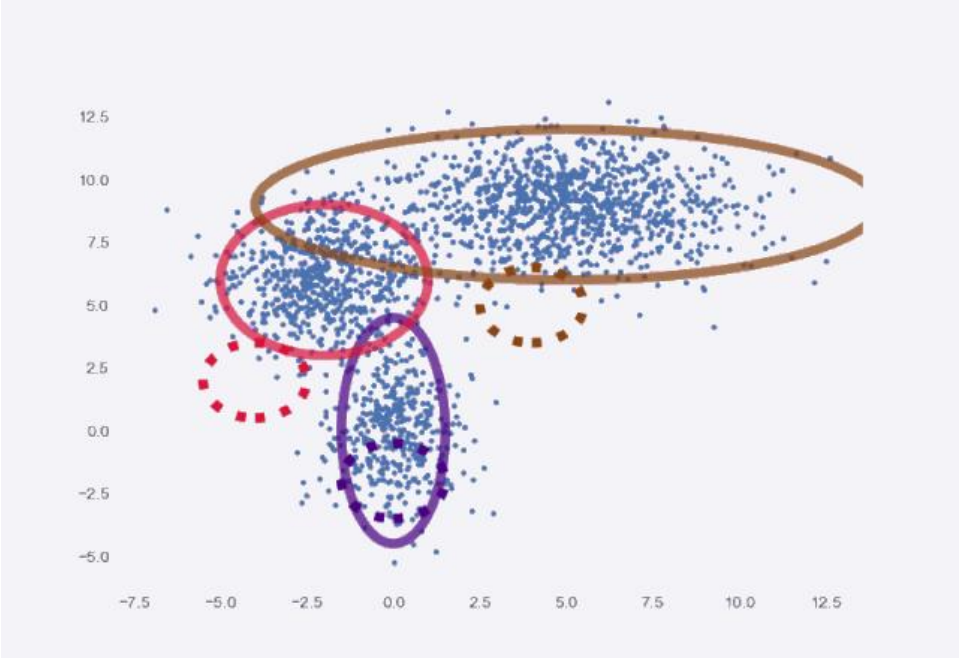
1.2 机器学习任务

- **聚类问题：目标空间是未知有限离散值空间，**

  $Z=X=\{x_1,x_2,\ldots,x_k\}$ 待求聚类函数，也称为聚类模型。

- ✓ 把已知数据集划分为不同子集（类别），不同类别之间的差距越大越好，同一类别内的数据差距越小越好。
- ✓ 聚类问题是无监督学习
- ✓ 常用的方法：
  - 均值聚类法K-Means 、
  - 基于密度聚类DBSCAN等





1.2 机器学习任务

- **关联问题：目标空间就是数据空间本身**

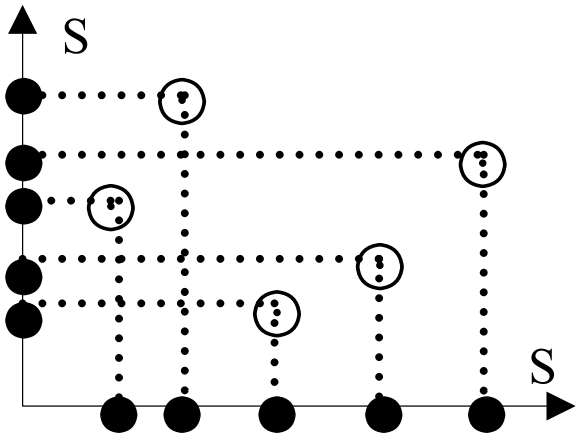
  Z=S,待求函数-求自身内部的一种映射。

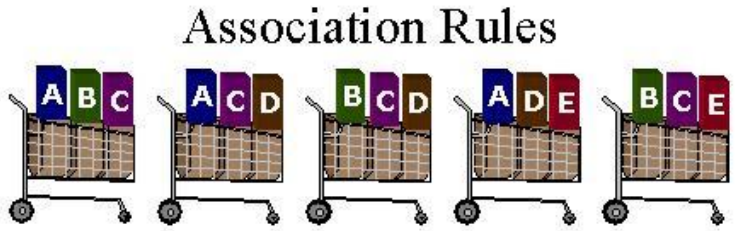
✓ 相关性分析或者关联问题

- 就是发现不同数据（属性）之间的相互依赖关系。
- 简单地说，就是可以从事物A推出事物B，即A→B

✓ 常用的方法：

- Apriori、贝叶斯网络等等。





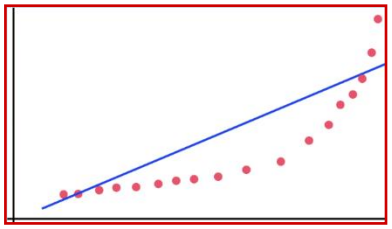
Rule	Support	Confidence
A ⇒ D	2/5	2/3
C ⇒ A	2/5	2/4
A ⇒ C	2/5	2/3
B & C ⇒ D	1/5	1/3

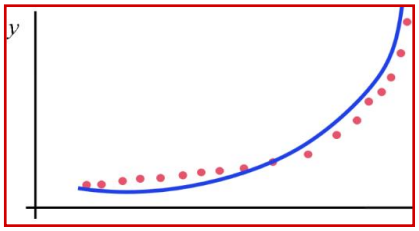
啤酒 => 尿布 [support=60%; confidence= 70%]

1.2 机器学习任务

- **机器学习设计 核心要素**

✓ 模型选择：线性/非线性…





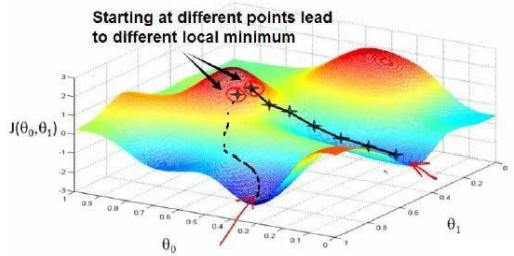
✓ 目标（损失）函数：误差平方/交叉熵…

$$J(\theta)=\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2 \qquad J(\theta)=-\frac{1}{N}\sum_{i=1}^{N}[y_i\log\hat{y}_i+(1-y_i)\log(1-\hat{y}_i)]$$

✓ 优化方法：梯度下降

$$\frac{\partial}{\partial\theta}J(\theta)=\frac{\partial}{\partial\theta}\frac{1}{2}\sum_{i=1}^{m}(h_\theta(x)-y)^2=(h_\theta(x)-y)x^{(i)}$$

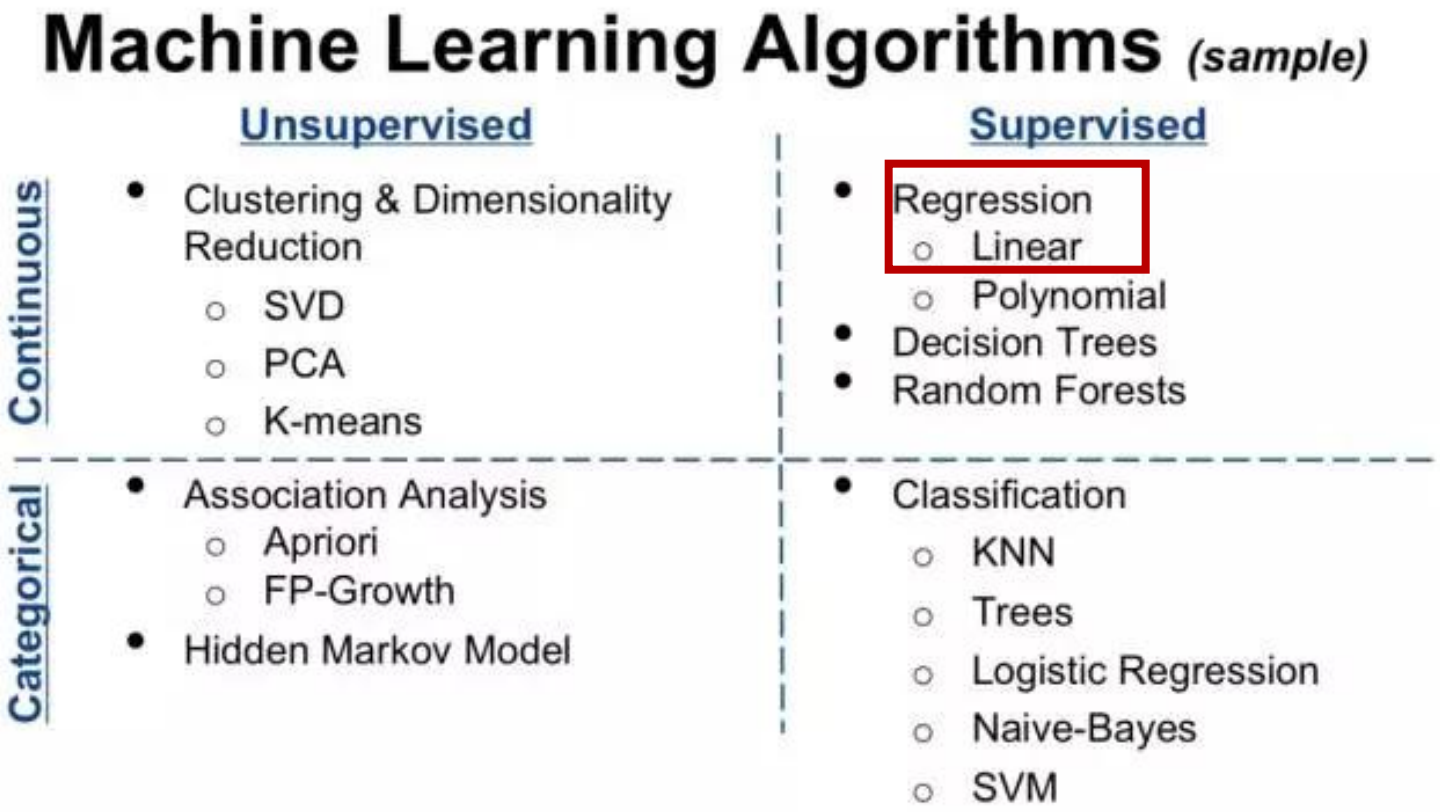
$$\theta_i=\theta_i-\alpha\frac{\partial}{\partial\theta}J(\theta)=\theta_i-\alpha(h_\theta(x)-y)x^{(i)}$$



如何合理设计？

1.2 机器学习任务

- 一个简单的例子：Regression



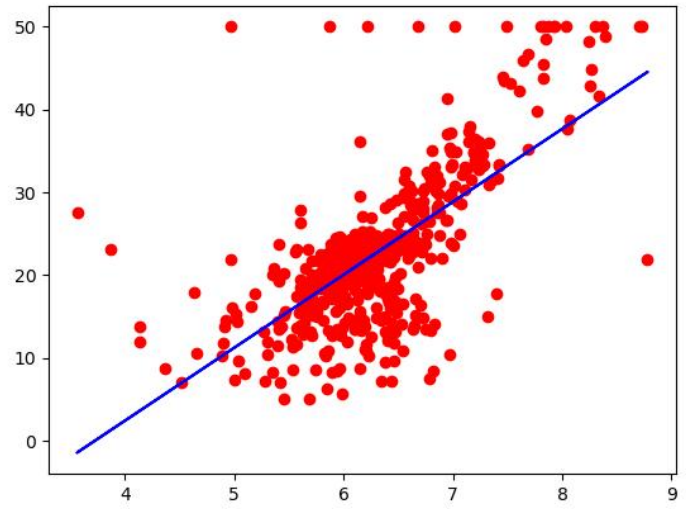
1.2 机器学习任务

- 一个简单的例子：Regression　　波士顿房价预测

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	Price
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33	36.2

数据集

CRIM：城镇人均犯罪率，百分比%
ZN：住宅用地所占比例，百分比%
INDUS：城镇中非住宅用地所占比例，百分比%
CHAS：CHAS 虚拟变量，用于回归分析
NOX：环保指数
RM：每栋住宅的房间数
AGE：1940 年以前建成的自住单位的比例，百分比%
DIS：距离 5 个波士顿的就业中心的加权距离
RAD：距离高速公路的便利指数
TAX：每一万美元的不动产税率,百分比%
PTRATIO：城镇中的教师学生比例，百分比%
B：关于黑人比例的一个参数
LSTAT：地区中有多少房东属于低收入人群，百分比%
MEDV：自住房屋房价中位数（也就是均价），单位 $1000 美元



1.2 机器学习任务

- **一个简单的例子：Regression** 波士顿房价预测

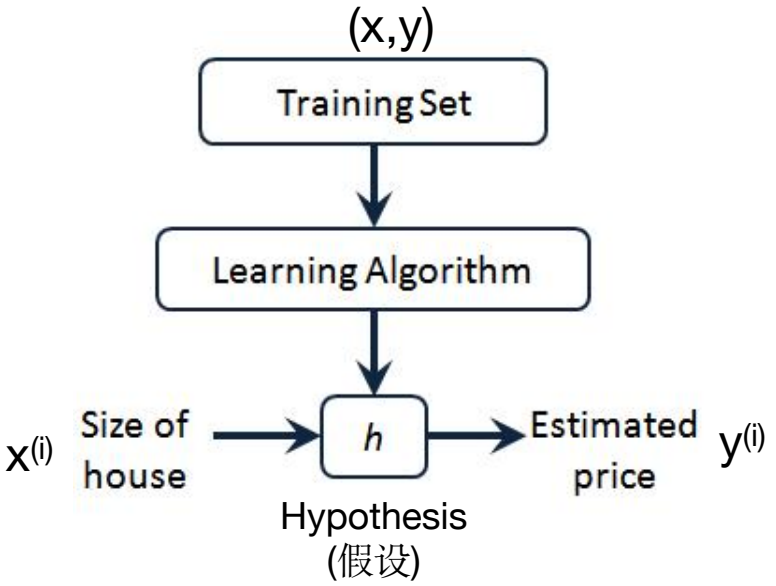
  - ✓ 模型选择：线性函数

$$h_\theta(x) = \theta_0 + \theta_1 x$$

  - ✓ 目标（损失）函数：平方误差

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

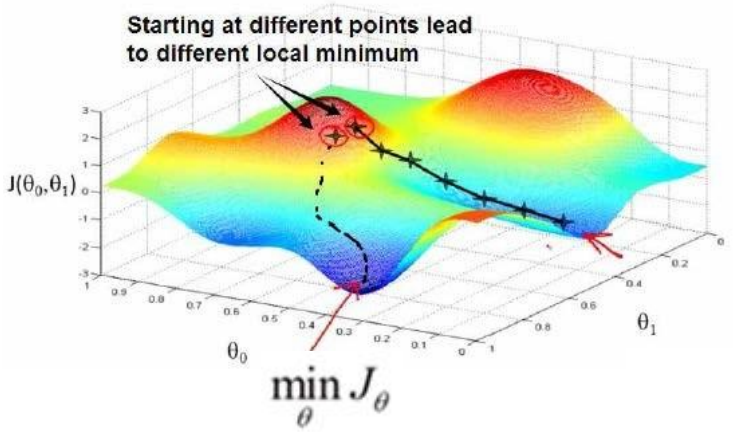
$$\min_\theta J_\theta$$



  - ✓ 优化方法：梯度下降

$$\frac{\partial}{\partial\theta}J(\theta) = \frac{\partial}{\partial\theta}\frac{1}{2}\sum_{i=1}^{m}(h_\theta(x) - y)^2 = (h_\theta(x) - y)x^{(i)}$$

$$\theta_i = \theta_i - \alpha\frac{\partial}{\partial\theta}J(\theta) = \theta_i - \alpha(h_\theta(x) - y)x^{(i)}$$



1.2 机器学习任务

- 一个简单的例子：Regression　　波士顿房价预测-线性回归实现

```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
from sklearn import datasets
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
```

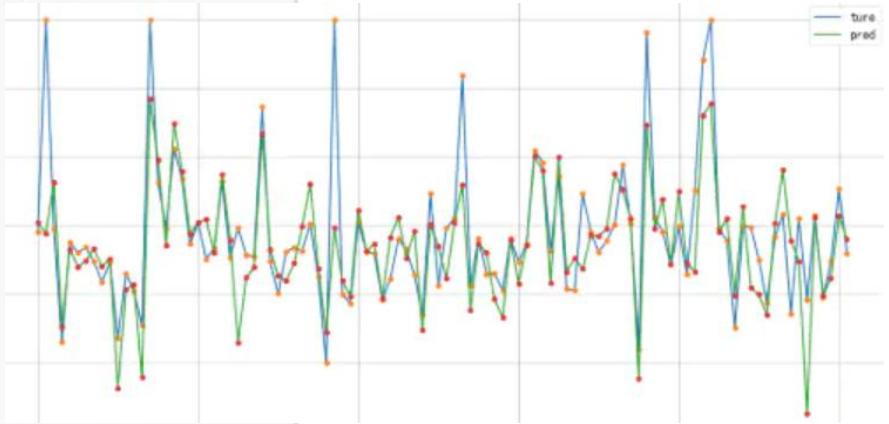
```
x = boston.data
y = boston.target
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,␣
 ↪random_state=0)

lr1 = LinearRegression()
lr1.fit(x_train, y_train)
y_train_pred = lr1.predict(x_train)
y_test_pred = lr1.predict(x_test)

print("The mean_sqared_error for train set is: {:.4f}".
 ↪format(mean_squared_error(y_train, y_train_pred)))
print("The mean_sqared_error for test set is: {:.4f}".
 ↪format(mean_squared_error(y_test, y_test_pred)))

print("The r2_core for train set is: {:.4f}".format(lr1.score(x_train,␣
 ↪y_train)))
print("The r2_core for test set is: {:.4f}".format(lr1.score(x_test, y_test)))

figure("      $R^2={:.4f}$".format(r2_score(y_test, y_test_pred)), [y_test,␣
 ↪"ture"], [y_test_pred, "pred"])
```



机器学习

1.1 机器学习概述

1.2 机器学习任务

1.3 机器学习评价

1.4 决策树模型及应用

1.3 机器学习评价

- **机器学习模型评价原则**
  - ✓ 模型学习结果的合理性和有效性

    模型的泛化能力（Generalization）越强越好

    评估指标：accuracy、precision、recall、f1-score
  - ✓ 算法复杂度（Complexity）

    评估指标：时间复杂度、空间复杂度等

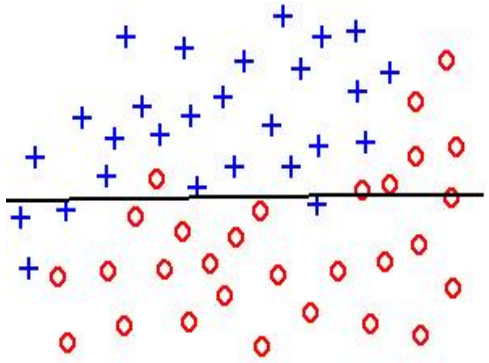
    减小时间复杂度常用的思路：
    - 简化问题，降低要求；
    - 用空间换时间；
    - 用并行化算法，提高并行度

1.3 机器学习评价

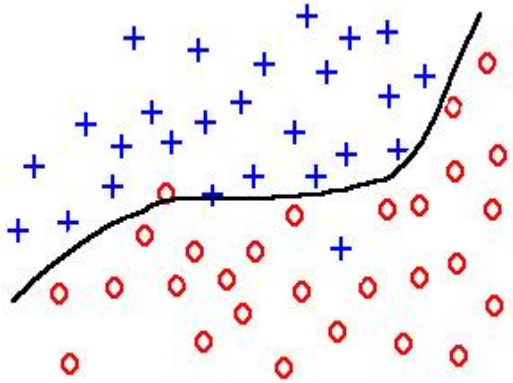
- **泛化能力（ Generalization ）理解？**

  泛化就是模型的稳定性，对新数据的预测能力。

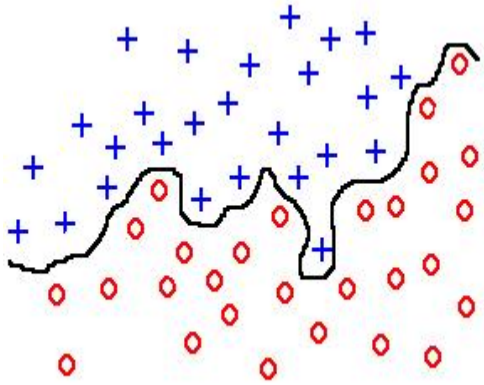
  经过训练的模型对于不是训练集的输入也能给出合适的输出。



低度拟合
underfitting



合适的
good fit



过度拟合
overfitting

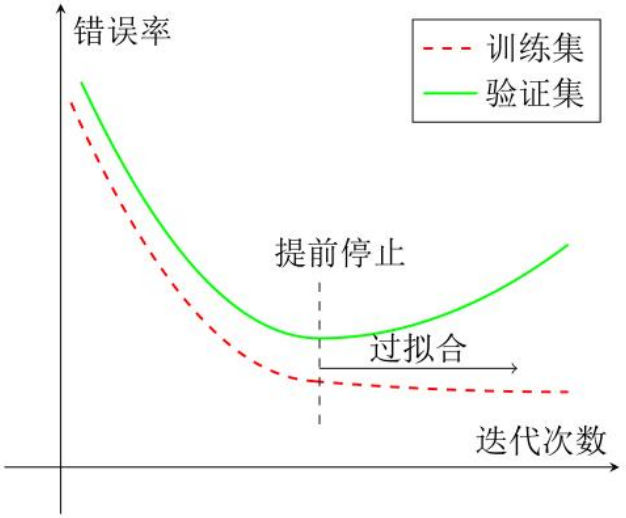
1.3 机器学习评价

- 如何提升模型的泛化能力？

$$L(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{2}\left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)}\right)^2$$

$$L(\mathbf{w}, b) + \frac{\lambda}{2}\|\mathbf{w}\|^2,$$

限制参数值范围来降低模型复杂性



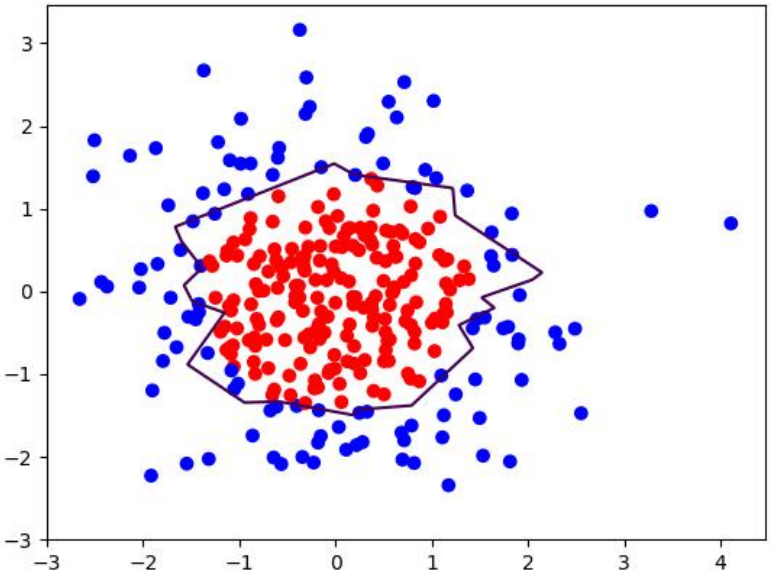
Regularization
加入正则化项。防止过拟合

Early stopping
提前停止，防止过拟合
从而提升泛化性

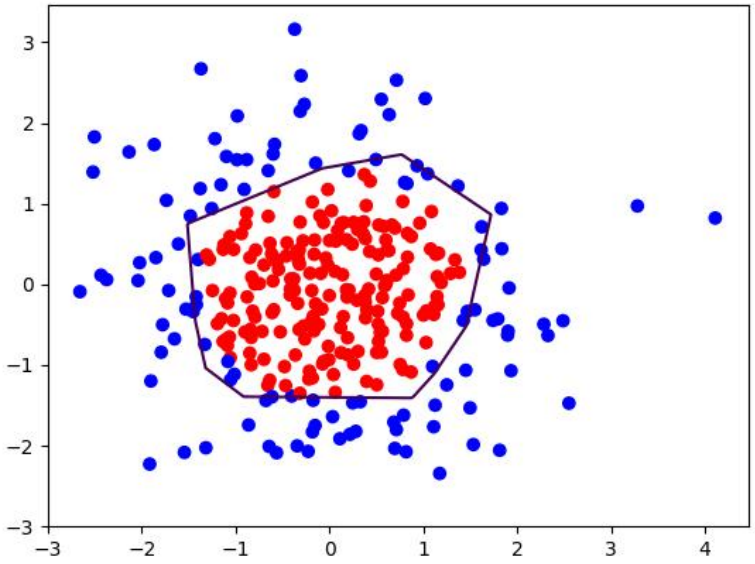
1.3 机器学习评价

- 如何提升模型的泛化能力？

$$L(\mathbf{w}, b) + \frac{\lambda}{2}\|\mathbf{w}\|^2,$$ L2正则化，减小参数值,W



未使用正则化



使用正则化

1.3 机器学习评价

- 如何提升模型的泛化能力？

泛化能力差的表现

训练集





数据来源: http://proceedings.mlr.press/v37/ganin15.pdf

测试集





99.5%

57.5%

机器学习的瓶颈，如何突破？



1.3 机器学习评价

- **如何提升模型的泛化能力？**

模型泛化能力提升方法:

(1)加入正则项;

(2)提前停止训练，防止过拟合;

(3)训练多个模型,然后以多个模型投票的结果做为最后结果(bagging);

(4)数据增强-根据数据集本身的结构特征，构造一些方法:

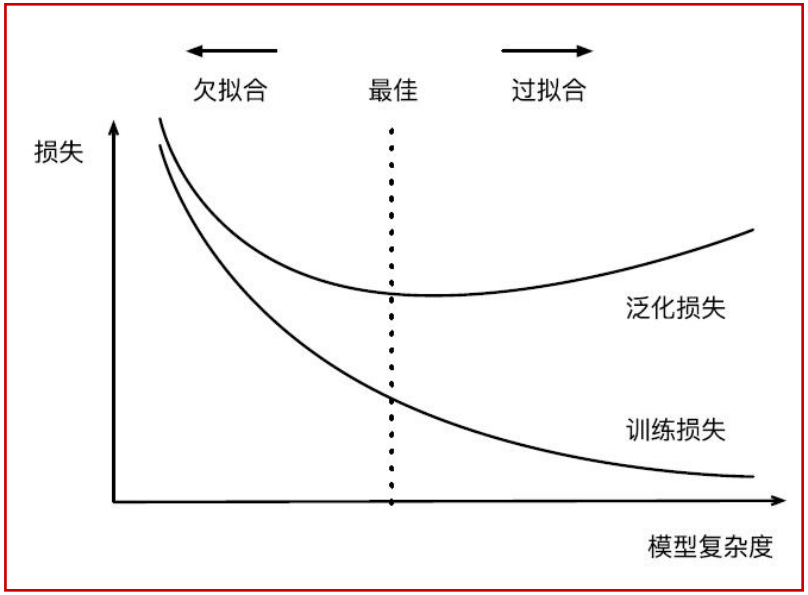
(a)特征提取，把多个特征合并为少数几个特征;

(b)根据已知的数据性质，构造新的样本，比如图像具有微旋转微平移不变性，那么就可以通过旋转和平移构建新样本;

(c)适当加入噪声;

1.3 机器学习评价

- **如何对泛化能力进行量化评价？**
  - ✓ 性能度量是衡量模型泛化能力的评价标准，反映了任务需求；
  - ✓ 使用不同的性能度量往往会导致不同的评判结果；
  - ✓ 什么模型是“好”的，不仅取决于算法和数据，还取决任务；



经验误差是否越小越好？

泛化误差越小越好！

泛化误差：在“未来”样本上的误差（未知的测试样本）

经验误差：在训练集上的误差，亦称“训练损失”

1.3 机器学习评价

- **泛化能力-评价的关键（测试集）**

  怎么获得“测试集”(test set)？ 测试集应该与训练集“互斥”
  避免数据泄露

  - 常见方法：
    - ✓ 留出法 (hold-out)
    - ✓ 交叉验证法 (cross validation)



- ➢ 保持数据分布一致性， 采样
- ➢ 多次重复划分，随机
- ➢ 测试集不能太大、不能太小

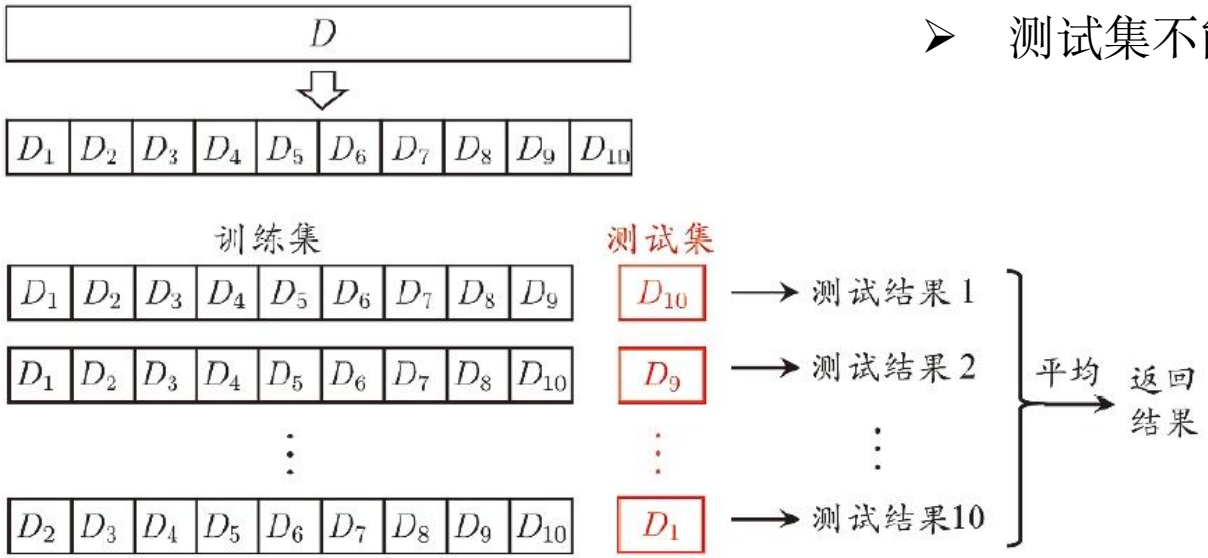


图 2.2 10 折交叉验证示意图

1.3 机器学习评价

- **泛化能力-评价的关键（调参）**
  - ✓ 算法的参数：一般由人工设定，称为“超参数”
  - ✓ 模型的参数：一般由学习确定，如神经网络权重
  - ✓ 参数调得好不好往往对最终性能有关键影响

Model Design

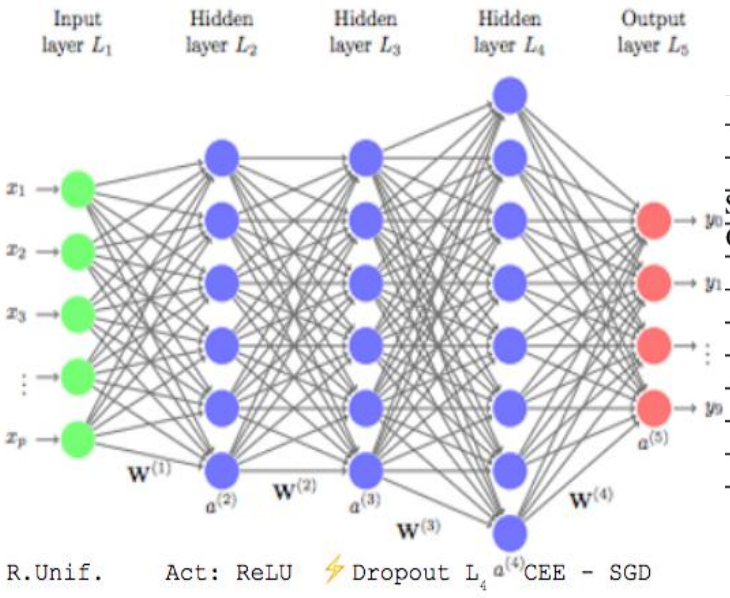
- Weight init.: Random Uniform
- Act.: ReLU
- Loss: CEE
- # Hidden Layers: 3
- # Units per layer {p, p+1, p+1, p+3, 10}
- Optimizer: SGD
- Dropout layer: L_4

Hyperparameters

- Learning rate
- Dropout Rate
- Batch size

Model Parameters

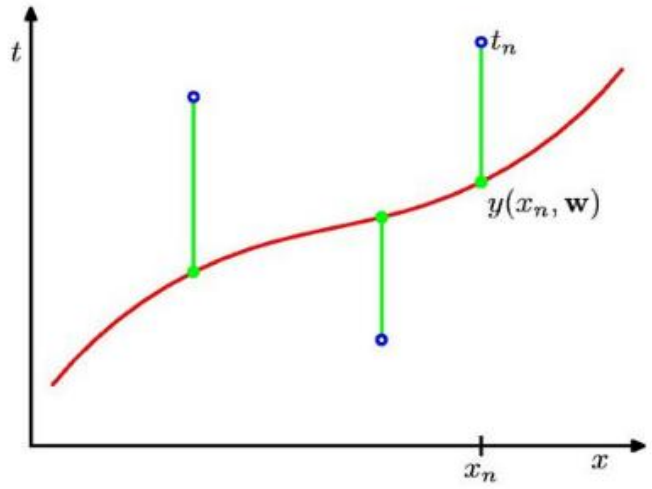
- $W^{(1)}$ => $W^{(4)}$



Parameter	Value
Optimizer	Adam
Loss Function	Cross Entropy
State Activation Function	Tanh
Gate Activation Function	Sigmoid
Initial Learning Rate	0.005
Learn Rate Drop Factor	0.2
Dropout	0.5
Gradient Threshold	1
Number of Layers	3
Time Step	64
Maximum Epochs	800
Minimum Batch Size	100

1.3 机器学习评价

- **性能度量-评价指标：误差（Error）**
  - ✓ 回归(regression) 任务常用均方误差：



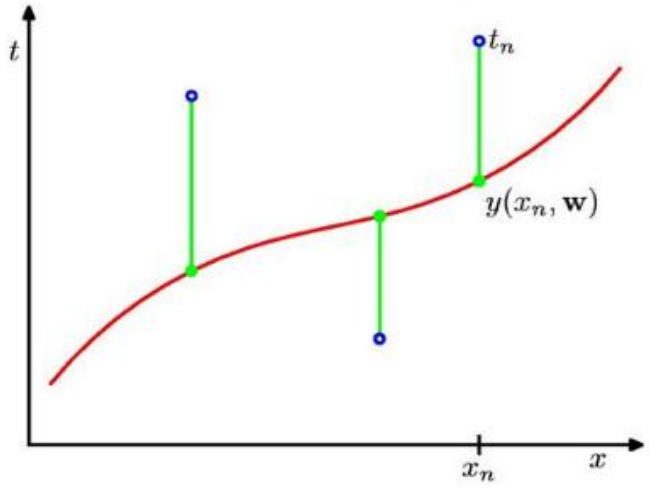
$$RMSE = \sqrt{\frac{1}{n}\sum\left(y_{observed} - y_{predicted}\right)^2}$$

$$MAPE = 100 \times \frac{1}{n}\sum\frac{\left|y_{observed} - y_{predicted}\right|}{y_{observed}}$$

$$MAE = \frac{1}{n}\sum\left|y_{observed} - y_{predicted}\right|$$

1.3 机器学习评价

- **性能度量-评价指标：误差（Error）**
  - ✓ 回归(regression) 任务常用均方误差：



$$RMSE = \sqrt{\frac{1}{n}\sum\left(y_{observed} - y_{predicted}\right)^2}$$ 关注较大误差

$$MAPE = 100 \times \frac{1}{n}\sum\frac{\left|y_{observed} - y_{predicted}\right|}{y_{observed}}$$ 关注相对误差

$$MAE = \frac{1}{n}\sum\left|y_{observed} - y_{predicted}\right|$$ 关注所有误差

当 RMSE（均方根误差）大于 MAE（平均绝对误差），这通常意味着数据中存在较大的误差或离群值。这是因为 RMSE 和 MAE 的计算方法导致它们对误差的惩罚程度不同。

1.3 机器学习评价

- **性能度量-评价指标：正确率或错误率**
  - ✓ 正确率是被正确处理的数据个数与所有被处理数据个数的比值

$$Accuracy(T) = \frac{|T_{Error<\varepsilon}|}{|T|}$$

其中$T_{Error}<\varepsilon$表示被正确处理的数据，也就是误差足够小的数据

  - ✓ 错误率则是没有被正确处理的数据个数与所有被处理数据个数的比值

$$ErrorRate(T) = \frac{|T| - |T_{Error<\varepsilon}|}{|T|} = 1 - \frac{|T_{Error<\varepsilon}|}{|T|} = 1 - Accuracy(T)$$

1.3 机器学习评价

- **性能度量-复合评价指标**
  - ✓ 准确率（Accurcay）
  - ✓ 精确率（Precision，也称为命中率、查准率）
  - ✓ 召回率（Recall，也称为查全率）
  - ✓ F值（F Measure，查准率和召回率的调和平均）

True Positive（TP）：将正样本预测为正样本
False Positive（FP）：将负样本预测为正样本
False Negative（FN）：将正样本预测为负样本
True Negative（TN）：将负样本预测为负样本

表 2.1 分类结果混淆矩阵

真实情况	预测结果	
	正例	反例
正例	TP（真正例）	FN（假反例）
反例	FP（假正例）	TN（真反例）

$$Accurcay = \frac{TP + TN}{TP + FP + FN + TN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

F1： precision和recall的调和平均

1.3 机器学习评价

表 2.1 分类结果混淆矩阵

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

- P表示算法预测这个样本为1（好西瓜）；
- N表示算法预测这个样本为0（坏西瓜）；
- T表示算法预测的和真实情况一样，即算法预测正确，
- F表示算法预测的和真实情况不一样，即算法预测不对。
- TP：正确地标记为正，即算法预测它为好西瓜，这个西瓜真实情况也是好西瓜（双重肯定是肯定）；
- FP：错误地标记为正，即算法预测它是好西瓜，但这个西瓜真实情况是坏西瓜；
- FN：错误地标记为负，即算法预测为坏西瓜，（F算法预测的不对）但这个西瓜真实情况是好西瓜（双重否定也是肯定）；
- TN：正确地标记为负，即算法标记为坏西瓜，（T算法预测的正确）这个西瓜真实情况是坏西瓜。

1.3 机器学习评价

$$recall = \frac{TP}{TP + FN}$$

所有实际为“好”的西瓜中，有多少被正确预测为“好”。

$$precision = \frac{TP}{TP + FP}$$

模型预测为“好”的西瓜中，有多少实际是“好”的。

$$F_1 = 2 \cdot \frac{precision \cdot recall}{precison + recall}$$

F_1分数是精确率和召回率的调和平均数，综合考虑了精确率和召回率，能够提供模型整体的评估。

$$accuracy = \frac{TP + TN}{TP + FN + TN + FP}$$

1.3 机器学习评价

	Predicted Good	Predicted Bad
Actual Good	40	10
Actual Bad	5	45

- **真正例（TP）**：40 (实际是好，预测为好)
- **假正例（FP）**：5 (实际是坏，预测为好)
- **假负例（FN）**：10 (实际是好，预测为坏)
- **真负例（TN）**：45 (实际是坏，预测为坏)

- **Precision**：

$$\text{Precision} = \frac{40}{40+5} = \frac{40}{45} = 0.89$$

- **Recall**：

$$\text{Recall} = \frac{40}{40+10} = \frac{40}{50} = 0.80$$

- **F1 分数**：

$$F1 = 2 \times \frac{0.89 \times 0.80}{0.89 + 0.80} = 2 \times \frac{0.712}{1.69} = 0.84$$

1.3 机器学习评价

- **关注召回率的领域**：医疗诊断、诈骗检测、灾难预警等，强调不能漏掉任何一个关键的正样本。
- **关注精确率的领域**：垃圾邮件检测、广告推荐、金融审批等，强调不能误报任何一个负样本为正样本。
- **关注F1分数的领域**：情感分析、恶意软件检测、推荐系统、语音识别等，要求在精确率和召回率之间取得平衡。

机器学习

1.1 机器学习概述

1.2 机器学习任务

1.3 机器学习评价

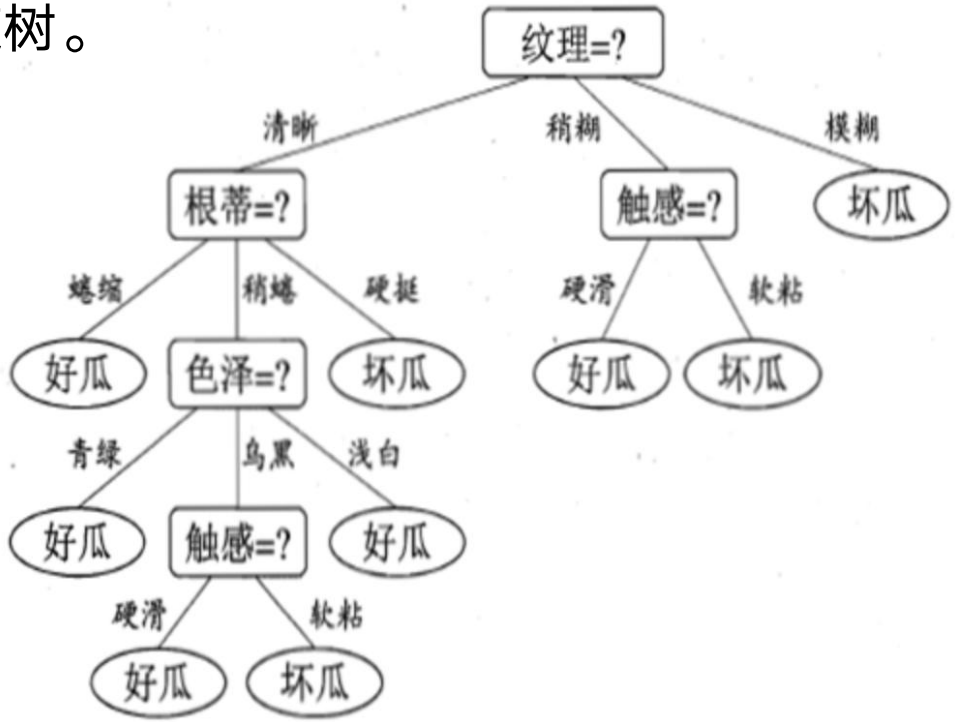
1.4 决策树模型及应用

1.4 决策树模型

决策树模型

- 决策树(Decision Tree) 是一种树形结构的监督学习模型，可用于分类(Classification)和回归(Regression)任务
- 它通过一系列“if-else”规则对输入特征进行递归划分，最终将样本分配到某个叶节点（类别标签或连续值）。
- 自顶向下递归方式构造决策树。

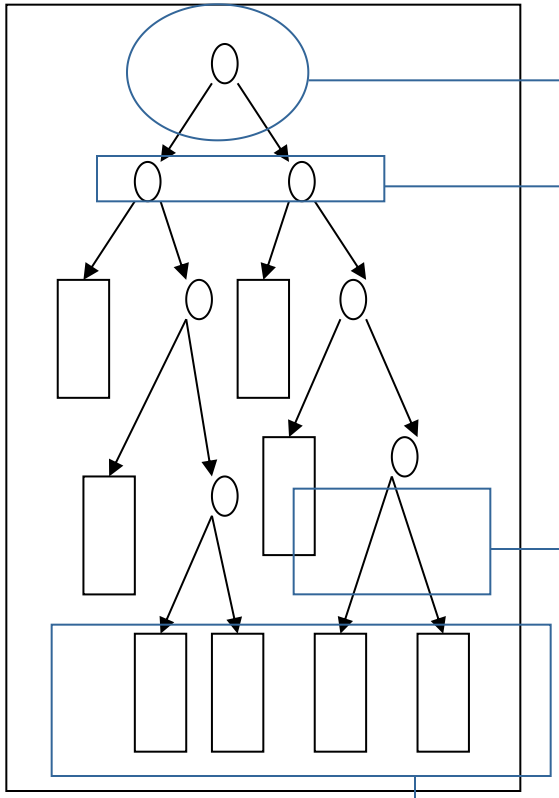




1.4 决策树模型实例

决策树模型

决策树算法以树状结构表示数据分类的结果。每个决策点实现一个具有离散输出的测试函数，记为分支。



根部节点(root node)

非叶子节点(non-leaf node)

(代表测试的条件，对数据属性的测试)

分支(branches)(代表测试的结果)

叶节点(leaf node)

(代表分类后所获得的分类标记)

决策树模型过程示例

1. **训练阶段**

从给定的训练数据集*DB*，构造出一棵决策树

class = DecisionTree(DB)

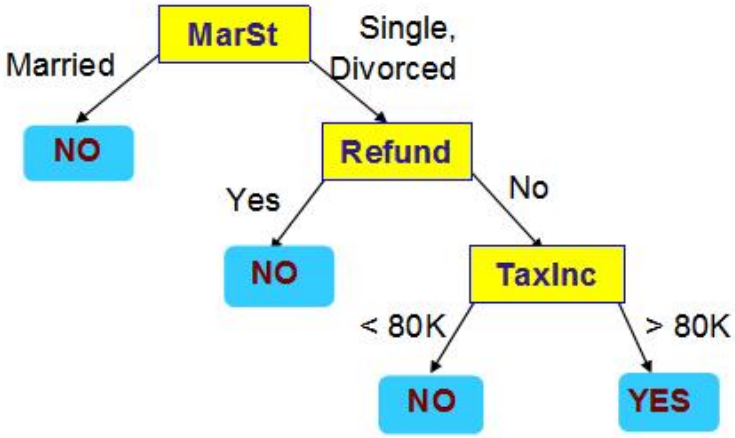
2. **分类阶段**

从根开始，按照决策树的分类属性逐层往下划分直到叶节点，获得决策（分类）结果

y = DecisionTree(x)

	categorical	categorical	continuous	class
Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

分类树结构唯一吗？

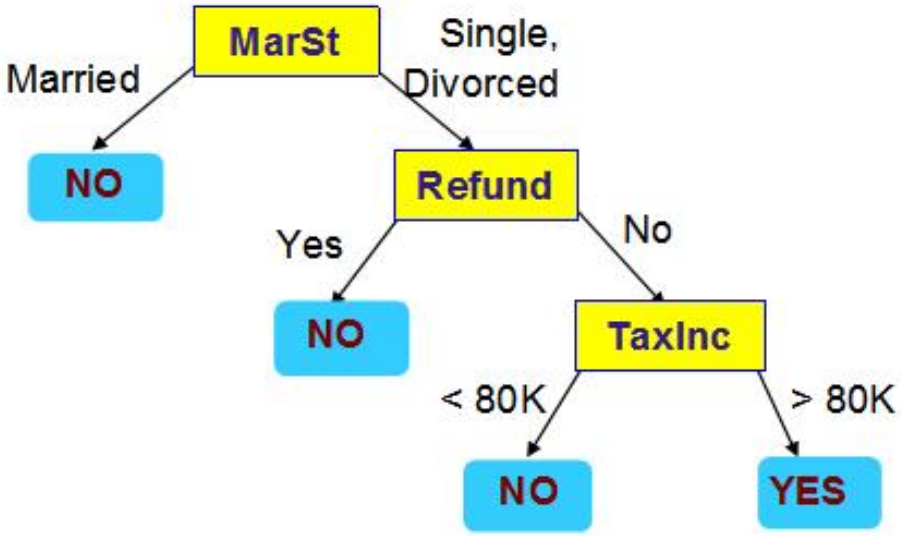


同一数据可能存在多个对应的决策树模型

决策树示例：训练阶段

- 例子: 判断是否存在欺诈行为

	categorical	categorical	continuous	class
Tid	**Refund**	**Marital Status**	**Taxable Income**	**Cheat**
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



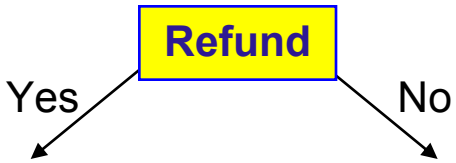
决策树模型

决策树示例：训练阶段

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

1. samples = { 1,2,3,4,5,6,7,8,9,10 }
attribute_list = {Refund, MarSt, TaxInc }

假设选择Refund为最优分割属性:



2. samples = { 1,4,7 }
attribute_list = { MarSt, TaxInc }

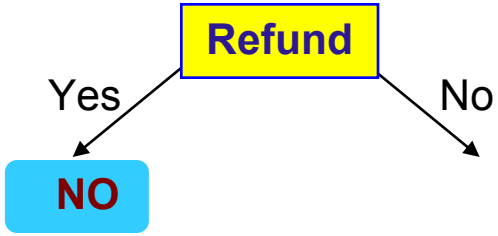
3. samples = {2,3,5,6,8,9,10 }
attribute_list = { MarSt, TaxInc }

决策树示例：训练阶段

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

2. samples = {1,4,7 }
attribute_list = { MarSt, TaxInc }

*samples*中所有样本属于同一个类Cheat=No

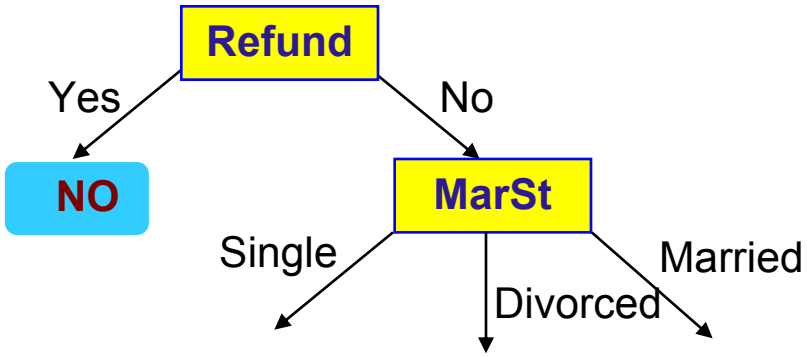


决策树示例：训练阶段

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

3. samples = { 2,3,5,6,8,9,10 }
attribute_list = { MarSt, TaxInc }

假设选择MarSt为最优分割属性:



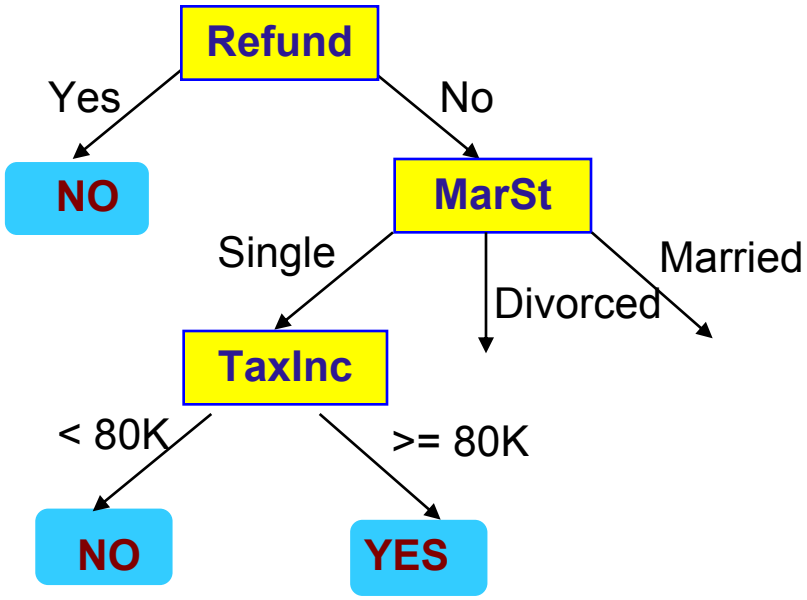
4. samples = { 3,8,10 }, attribute_list = {TaxInc}
5. samples = { 5,7 }, attribute_list = {TaxInc}
6. samples = { 2,9 }, attribute_list = {TaxInc}

决策树示例：训练阶段

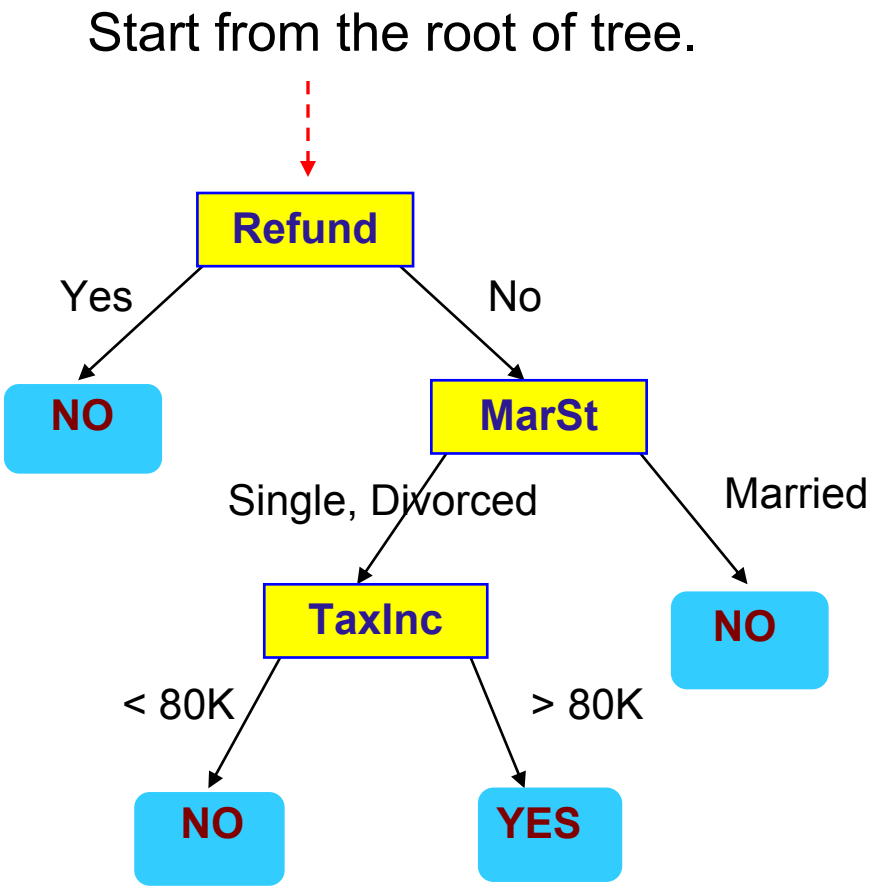
Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

4. samples = { 3,8,10 }
attribute_list = { TaxInc }

选择TaxInc为最优分割属性:



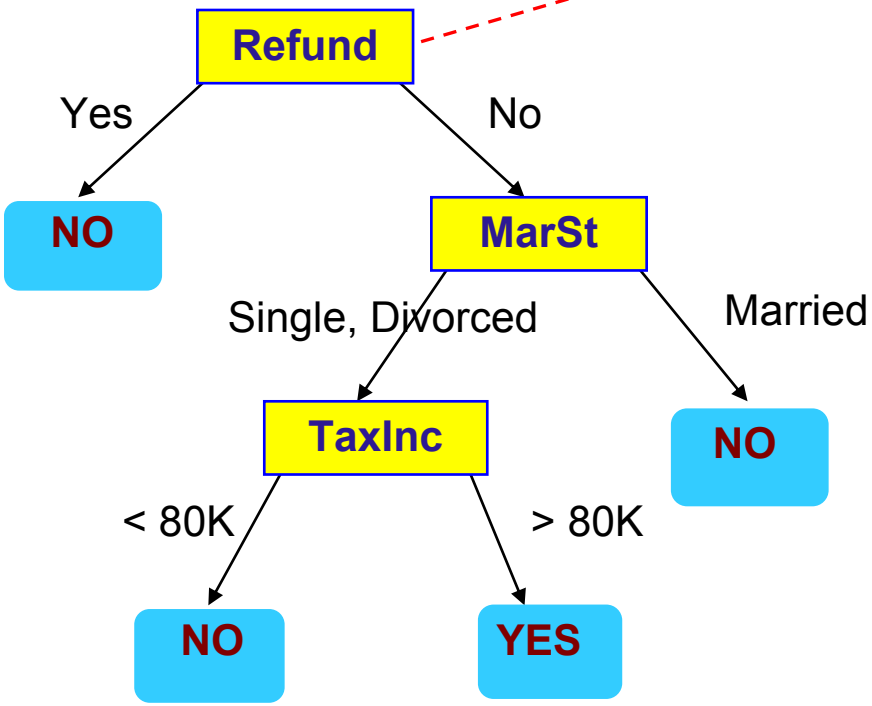
决策树示例：分类阶段



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

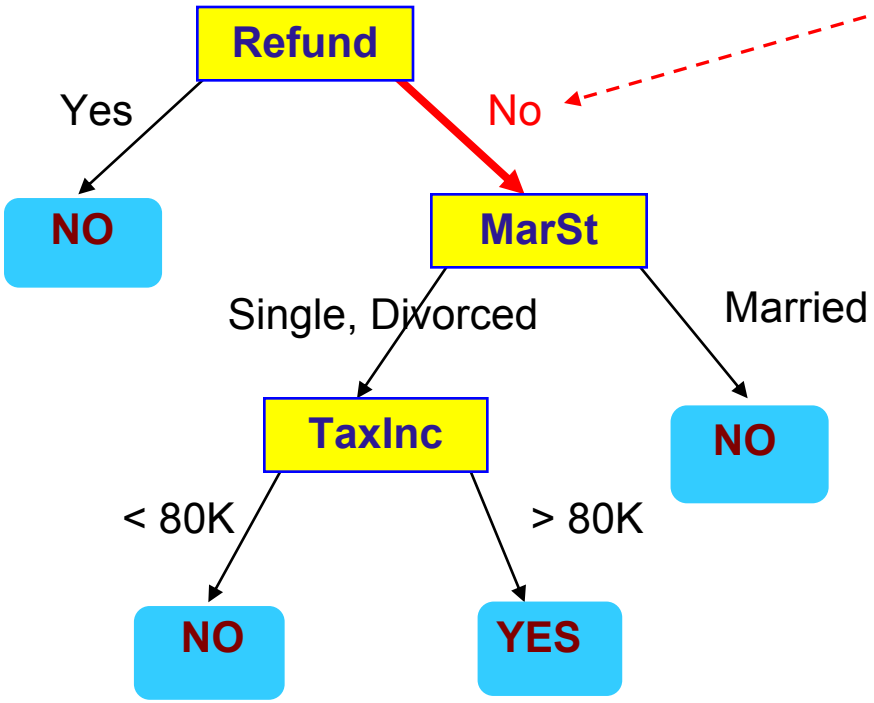
决策树示例：分类阶段



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

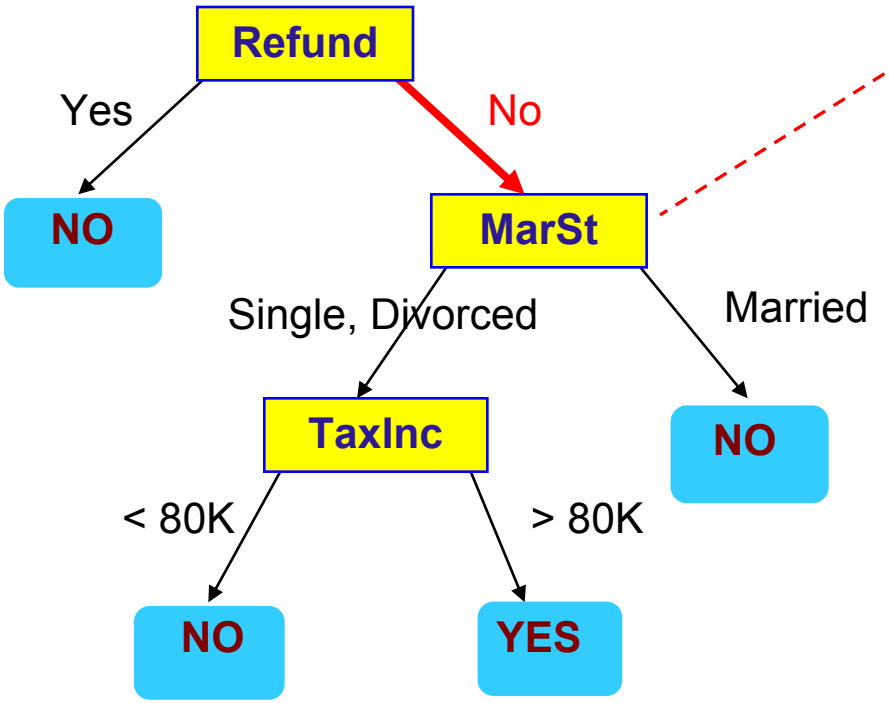
决策树示例：分类阶段



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

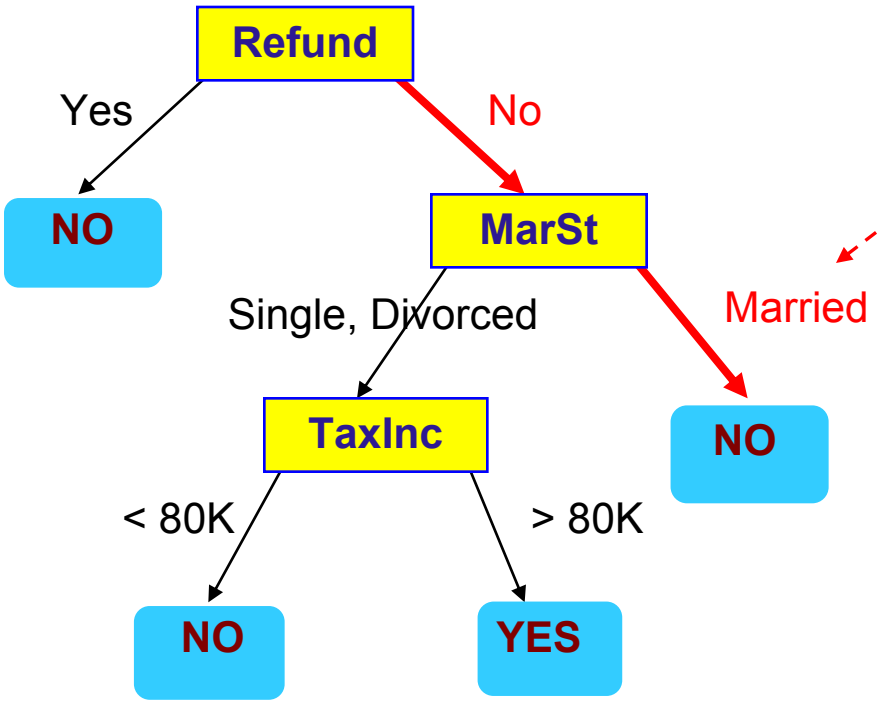
决策树示例：分类阶段



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

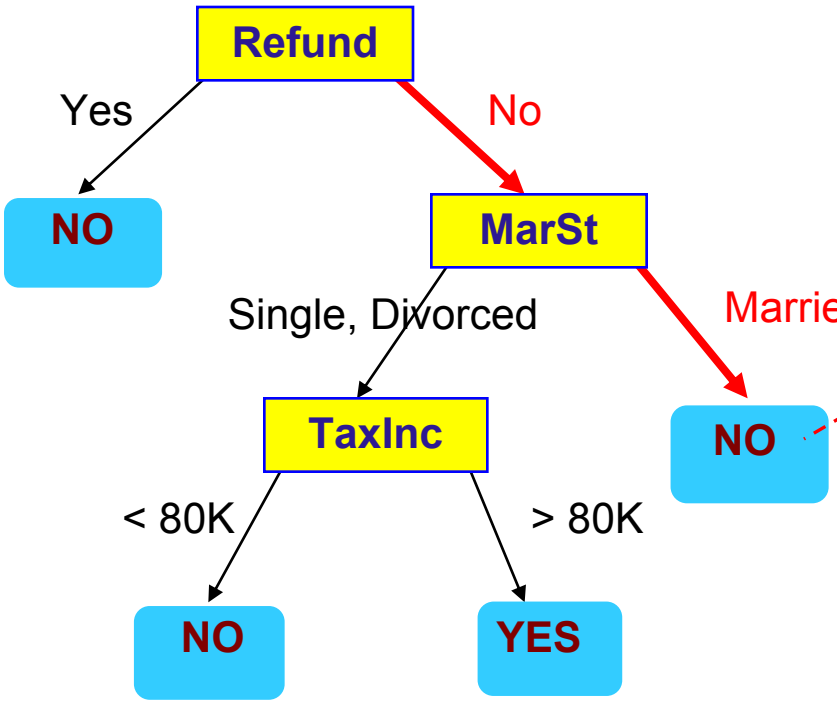
决策树示例：分类阶段



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

决策树示例：分类阶段



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

Assign Cheat to “No”

1.4 决策树模型

- **决策树模型原理**

  策略：“分而治之”(divide-and-conquer)

  自顶向下递归方式构造决策树，即 自根至叶的递归过程

  在每个中间结点寻找一个“划分”(split or test)属性

  三种停止条件：

  (1) 当前结点包含的样本全属于同一类别，无需划分;

  (2) 当前属性集为空, 或是所有样本在所有属性上取值相同，无法划分;

  (3) 当前结点包含的样本集合为空，不能划分。

- 决策树归纳的基本算法是**贪心算法**

  贪心算法：在每一步选择中都采取在当前状态下最好/优的选择。

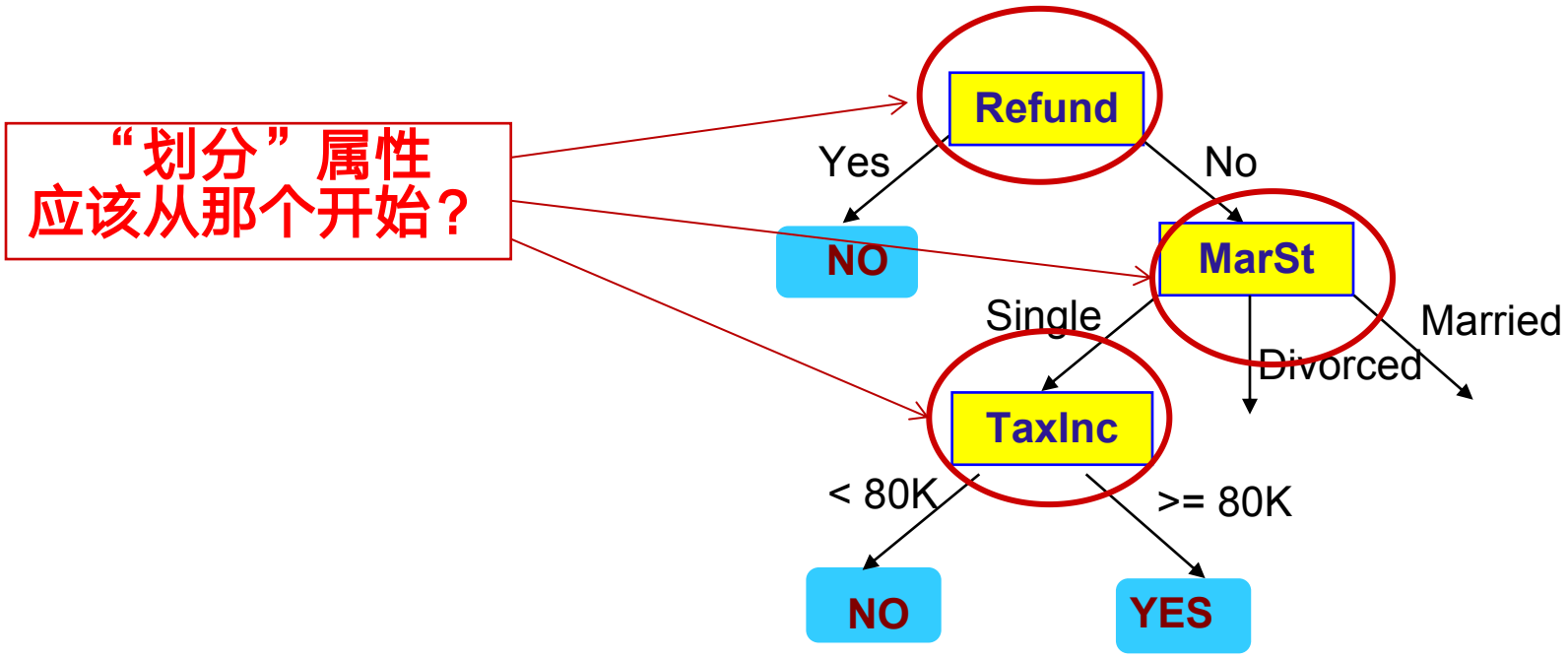
  在其生成过程中，分割方法即**属性选择度量**是关键。

决策树分类的关键

策略：“分而治之”(divide-and-conquer)

自根至叶的递归过程

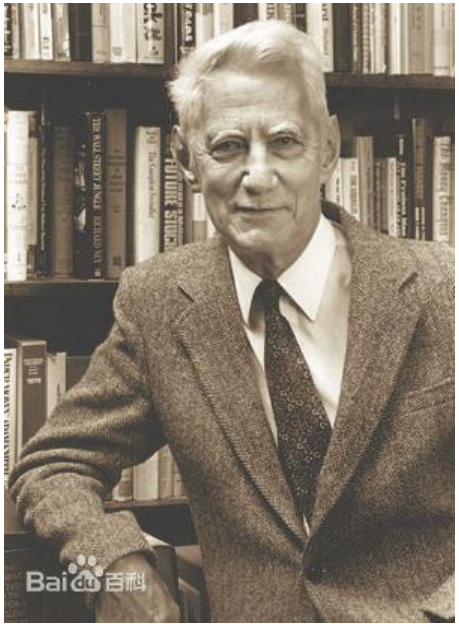
在每个中间结点寻找一个“划分”(split or test)属性



1.4 决策树分类的关键

- 信息论创始人——香农

“划分”属性的选择？引入熵理论



- 克劳德·艾尔伍德·香农（Claude Elwood Shannon，1916年4月30日—2001年2月24日）是美国数学家、信息论的创始人。
- 香农提出了信息熵的概念，为信息论和数字通信奠定了基础。
- 他是使我们的世界能进行即时通信的少数科学家和思想家之一。他是美国科学院院士、美国工程院院士、英国皇家学会会员、美国哲学学会会员。他获得过许多荣誉和奖励。

1.4 信息熵

- 物质世界的三大支柱：物质、能量和信息
  物质、能量和信息如何度量?

  - 物质质量的度量：通过单位为克、千克等
  - 能量（热量、功）的度量：通过“卡、焦耳”等新单位
  - 然而，关于文字、数字、图画、声音的知识已有几千年历史了。但是它们的总称是什么，它们如何统一地计量？

1.4 信息熵

- 熵(entropy)
  - 描述物质系统状态:该状态可能出现的程度。
  - 平均信息量

    若一个系统中存在多个事件$E_1, E_2, \cdots E_n$

    每个事件出现的概率是$p_1, p_2, \cdots p_n$

    则这个系统的平均信息量是:

    如果log以 2 为底，计算出的就是 bits

$$\text{entropy}(p_1, p_2, \ldots, p_n) = -p_1 \log_2 p_1 - p_2 \log_2 p_2 \ldots - p_n \log_2 p_n$$

$$Entropy(D) = \sum_{i=1}^{c} -P_i \log_2 (P_i)$$

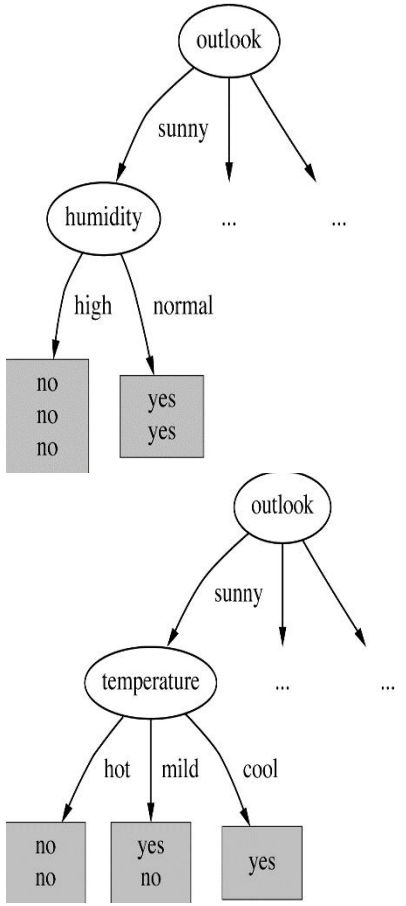
“熵”：系统的混乱程度!

熵与信息内容的不确定程度有等价关系

决策树分类的关键

- **系统越无序、越混乱，熵就越大。**
- **构造决策树，得到尽可能纯的节点。**

- 如果一个节点上的数据类值在可能的类值上均匀分布，则称节点的熵（无序性）最大。
- 选择一个属性划分数据，使得子女节点上数据的类值（例中“yes”或“no”）大部分都相同（低无序性）。
- 如果一个节点上的数据的类值对于所有数据都相同，则熵最小。



通过分裂，得到尽可能纯（类值对于所有数据都相同）的节点。
构建决策树相当于降低系统的熵。

决策树分类的关键

- **熵是度量样本集合“纯度”最常用的一种指标**

假定当前样本集合 D 中第 k 类样本所占的比例为p_k，则 D 的**信息熵**定义为：

$$Ent(D) = -\sum_{k=1}^{|y|} p_k \log_2 \left(p_k \right)$$

Ent(D) 的值越小，则D的纯度越高

- **根据划分前后熵变化的不同衡量函数**，决策树主要分为三种算法
  - ID3

    任务类型：分类　　划分准则：信息增益
  - C4.5

    任务类型：分类　　划分准则：信息增益率
  - CART

    任务类型：分类/回归　划分准则：基尼不纯度/MSE+MAE

决策树分类的关键

- **以ID3算法为例：**
  - **信息增益计算**

属性 a 的取值：$\{a^1, a^2, \ldots, a^V\}$　D^v：D 中在 a 上取值 $= a^v$ 的样本集合

以属性 a 对数据集 D 进行划分所获得的**信息增益**为：

$$\mathrm{Gain}(D, a) = \mathrm{Ent}(D) - \sum_{v=1}^{V} \frac{|D^v|}{|D|} \mathrm{Ent}(D^v)$$

划分前的信息熵　　划分后的信息熵

第 v 个分支的权重,样本越多越重要

选择获得最大（OR最小）信息增益的属性进行划分？

决策树伪码（经典ID3算法）

输入： 训练集 $D = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_m, y_m)\}$;
属性集 $A = \{a_1, a_2, \ldots, a_d\}$.

过程： 函数 TreeGenerate(D, A)

```
1: 生成结点 node;
2: if D 中样本全属于同一类别 C then
3:     将 node 标记为 C 类叶结点; return
4: end if
5: if A = ∅ OR D 中样本在 A 上取值相同 then
6:     将 node 标记为叶结点, 其类别标记为 D 中样本数最多的类; return
7: end if
8: 从 A 中选择最优划分属性 a_*;
9: for a_* 的每一个值 a_*^v do
10:    为 node 生成一个分支; 令 D_v 表示 D 中在 a_* 上取值为 a_*^v 的样本子集;
11:    if D_v 为空 then
12:        将分支结点标记为叶结点, 其类别标记为 D 中样本最多的类; return
13:    else
14:        以 TreeGenerate(D_v, A \ {a_*})为分支结点
15:    end if
16: end for
```

输出： 以 node 为根结点的一棵决策树

决策树伪码（经典ID3算法）

输入： 训练集 $D = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_m, y_m)\}$;
属性集 $A = \{a_1, a_2, \ldots, a_d\}$.

过程： 函数 TreeGenerate(D, A)

```
1:  生成结点 node;
2:  if D 中样本全属于同一类别 C then
3:     将 node 标记为 C 类叶结点; return
4:  end if
5:  if A = ∅ OR D 中样本在 A 上取值相同 then
6:     将 node 标记为叶结点, 其类别标记为 D 中样本数最多的类; return
7:  end if
8:  从 A 中选择最优划分属性 a_*;
9:  for a_* 的每一个值 a_*^v do
10:    为 node 生成一个分支; 令 D_v 表示 D 中在 a_* 上取值为 a_*^v 的样本子集;
11:    if D_v 为空 then
12:       将分支结点标记为叶结点, 其类别标记为 D 中样本最多的类; return
13:    else
14:       以 TreeGenerate(D_v, A \ {a_*})为分支结点
15:    end if
16: end for
```

输出： 以 node 为根结点的一棵决策树

递归返回，停止条件(1)

递归返回，停止条件(2)

递归返回，停止条件(3)

停止条件

（1）当前结点包含的样本全属于同一类别，无需划分;

（2）当前属性集为空，或是所有样本在所有属性上取值相同，无法划分;

（3）当前结点包含的样本集合为空，不能划分。

决策树伪码（经典ID3算法）

输入： 训练集 $D = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_m, y_m)\}$;
属性集 $A = \{a_1, a_2, \ldots, a_d\}$.

过程： 函数 TreeGenerate(D, A)

```
1:  生成结点 node;
2:  if D 中样本全属于同一类别 C then
3:     将 node 标记为 C 类叶结点; return
4:  end if
5:  if A = ∅ OR D 中样本在 A 上取值相同 then
6:     将 node 标记为叶结点, 其类别标记为 D 中样本数最多的类; return
7:  end if
8:  从 A 中选择最优划分属性 a_*;
9:  for a_* 的每一个值 a_*^v do
10:    为 node 生成一个分支; 令 D_v 表示 D 中在 a_* 上取值为 a_*^v 的样本子集;
11:    if D_v 为空 then
12:       将分支结点标记为叶结点, 其类别标记为 D 中样本最多的类; return
13:    else
14:       以 TreeGenerate(D_v, A \ {a_*})为分支结点
15:    end if
16: end for
```

输出： 以 node 为根结点的一棵决策树

递归返回，停止条件(1)

递归返回，停止条件(2)

决策树核心》选择获得最大**信息增益**的属性进行划分

递归返回，停止条件(3)

停止条件

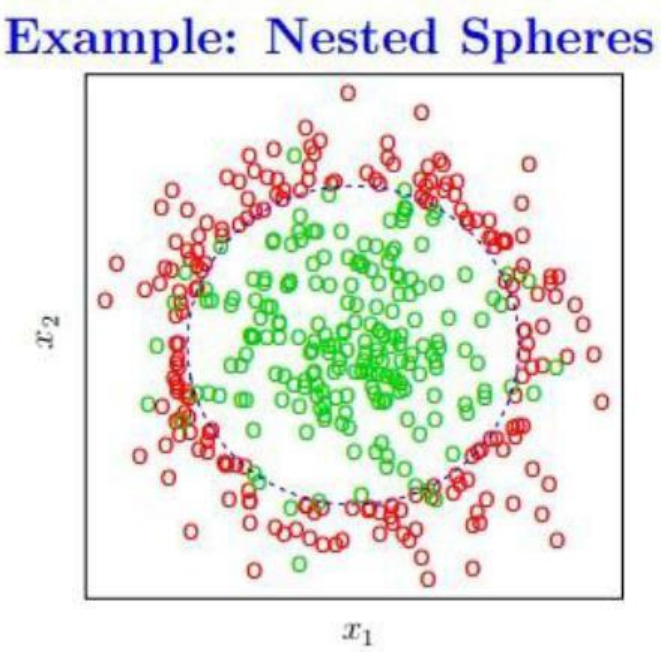
（1）当前结点包含的样本全属于同一类别，无需划分;

（2）当前属性集为空，或是所有样本在所有属性上取值相同，无法划分;

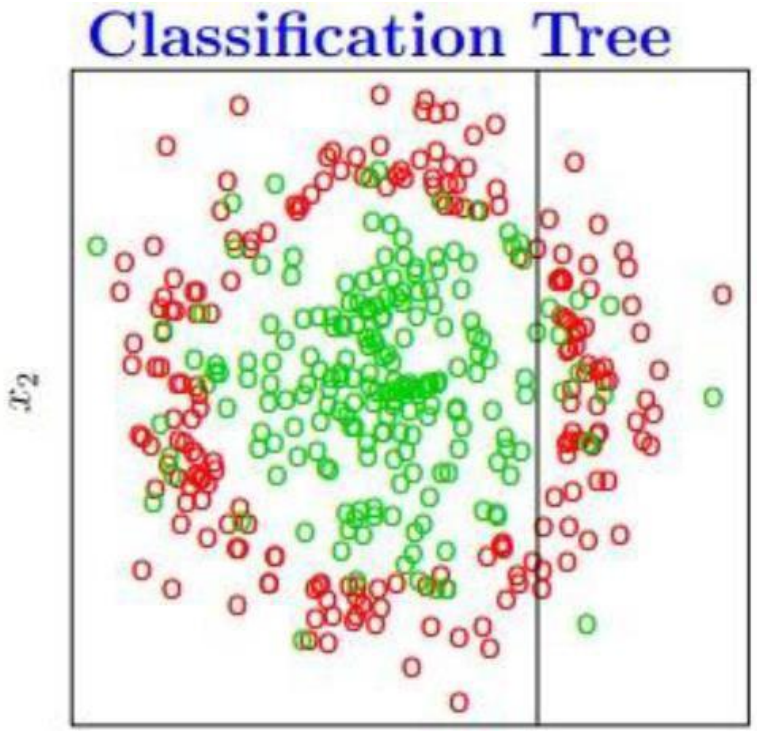
（3）当前结点包含的样本集合为空，不能划分。

1.4 决策树的生成过程

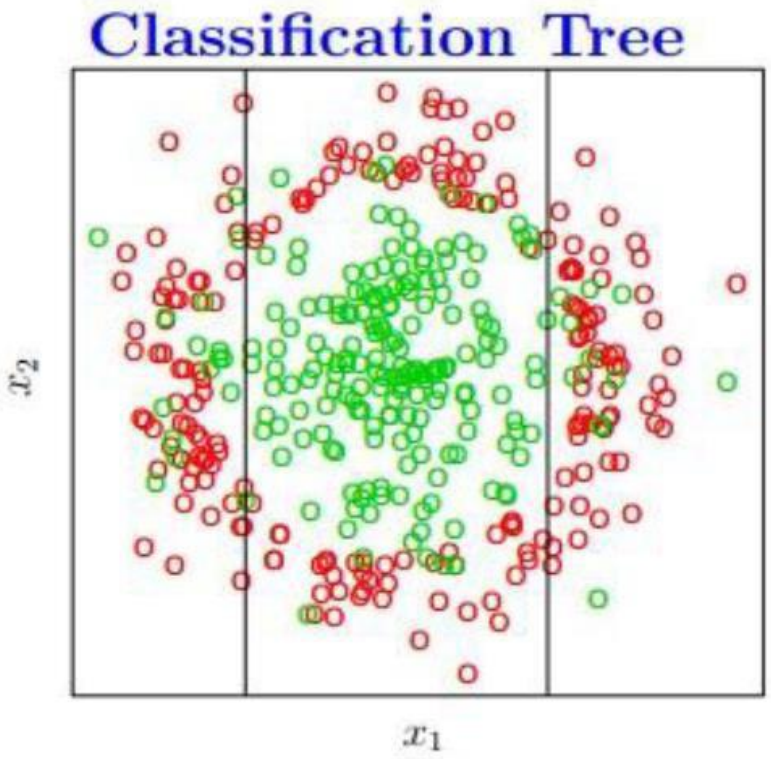
- 对于下面的数据，希望分割成红色和绿色两个类



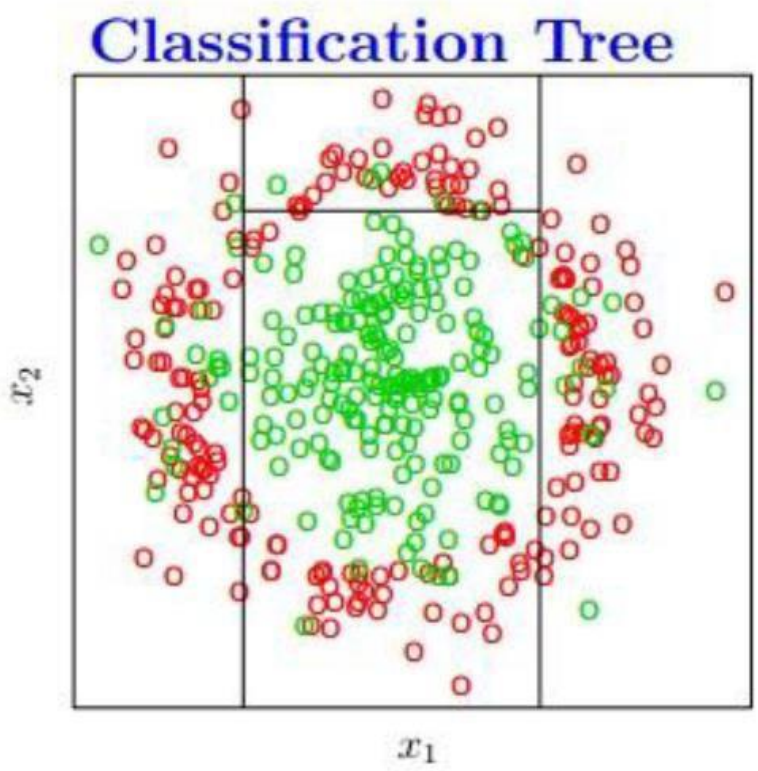
1.4 决策树的生成过程



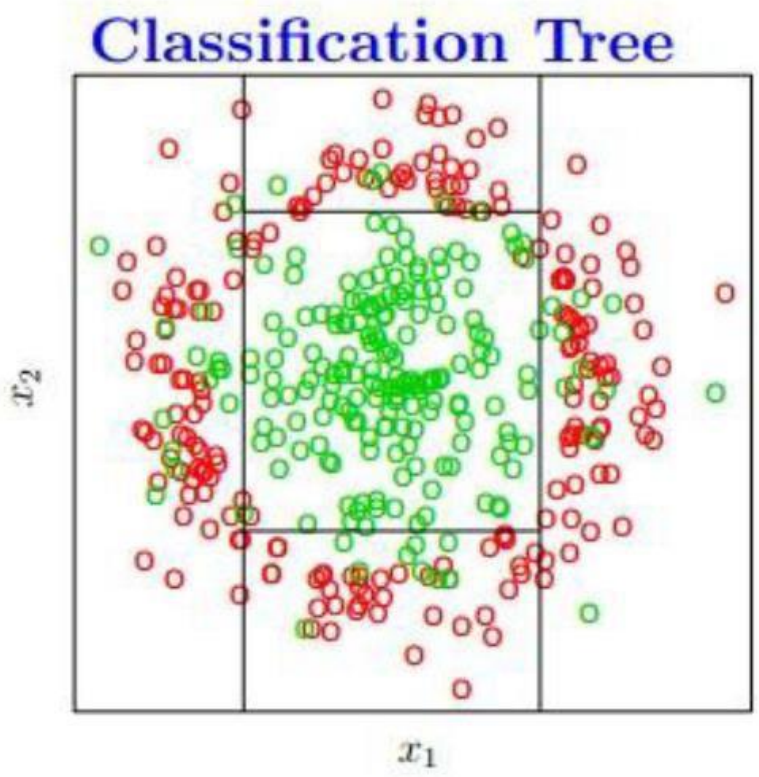
1.4 决策树的生成过程



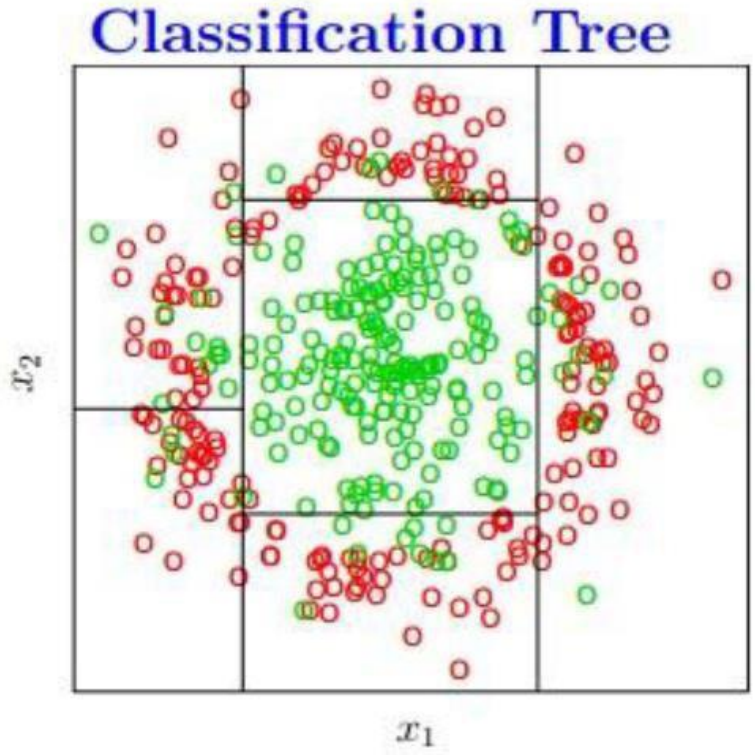
1.4 决策树的生成过程



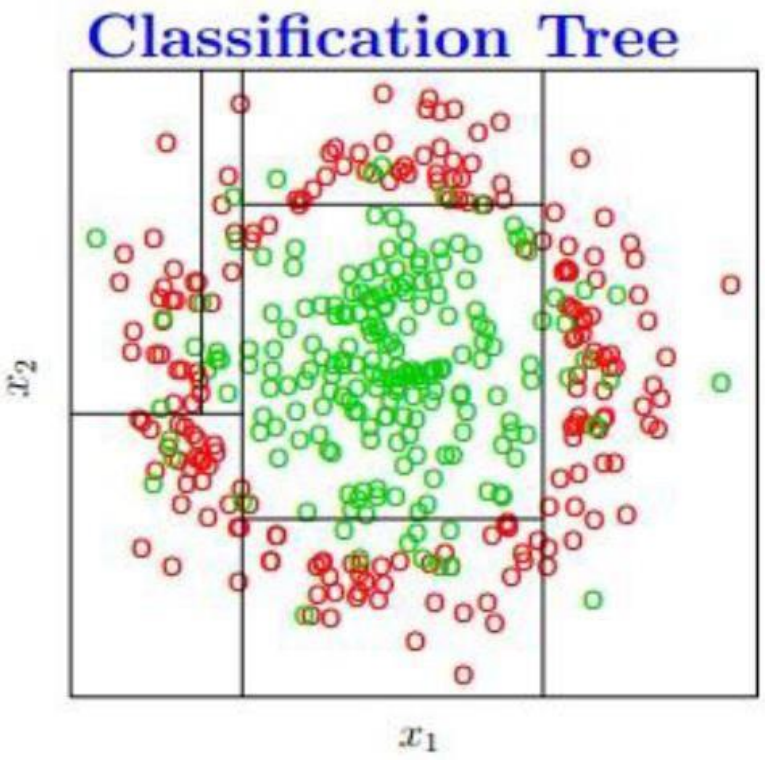
1.4 决策树的生成过程



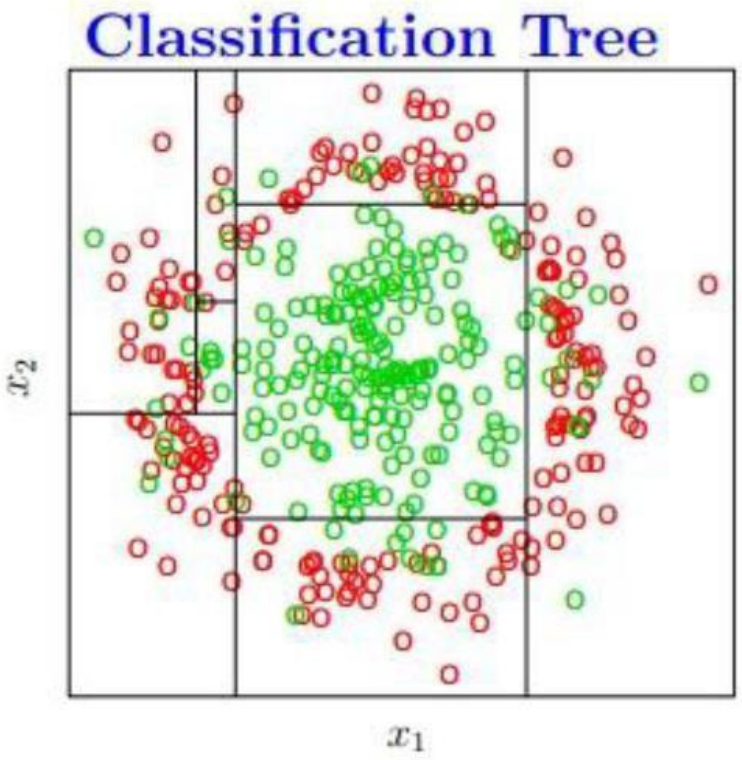
1.4 决策树的生成过程



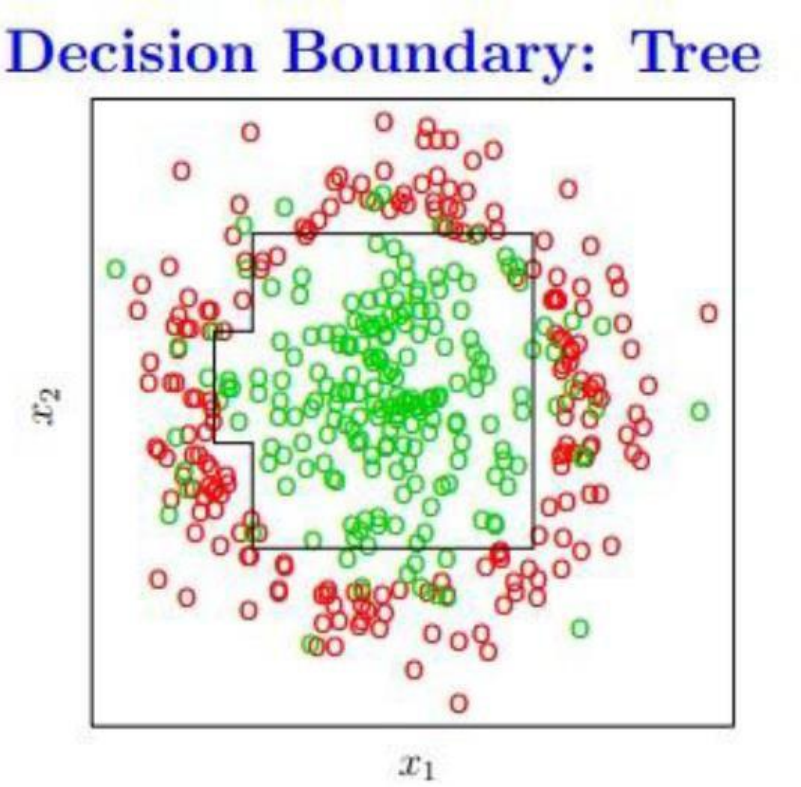
1.4 决策树的生成过程



1.4 决策树的生成过程



1.4 决策树的生成过程



决策树算法实例

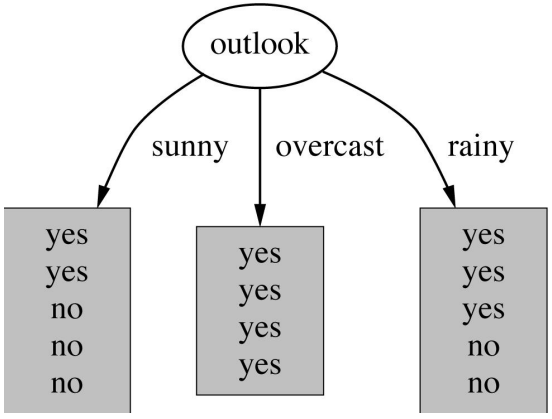
- 基于气象数据集（如右图）

什么因素影响是否去打网球？

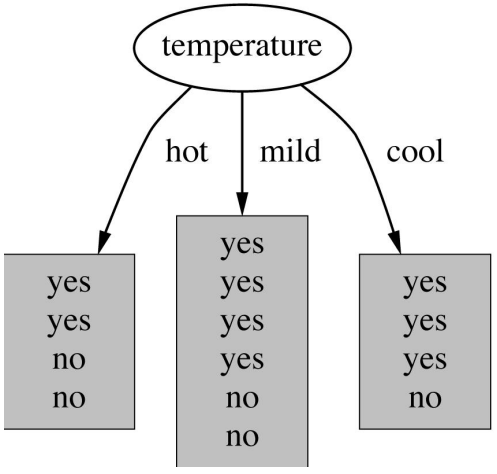
Outlook	Temperature	Humidity	Windy	Play?
sunny	hot	high	false	No
sunny	hot	high	true	No
overcast	hot	high	false	Yes
rain	mild	high	false	Yes
rain	cool	normal	false	Yes
rain	cool	normal	true	No
overcast	cool	normal	true	Yes
sunny	mild	high	false	No
sunny	cool	normal	false	Yes
rain	mild	normal	false	Yes
sunny	mild	normal	true	Yes
overcast	mild	high	true	Yes
overcast	hot	normal	false	Yes
rain	mild	high	true	No

决策划分

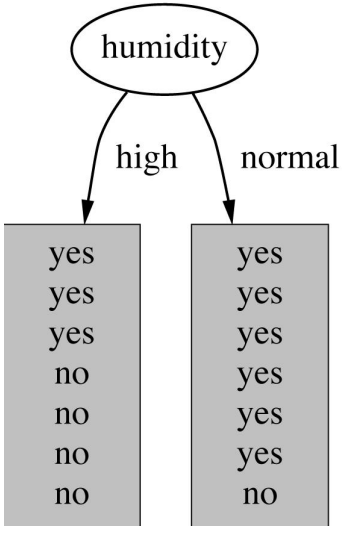
1.基于天气的划分



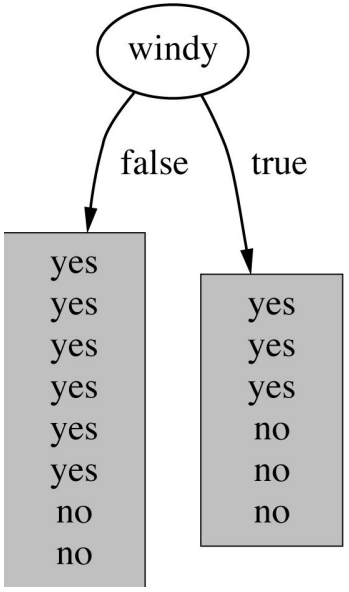
2.基于温度的划分



3.基于湿度的划分



4.基于有风的划分



构造决策树

（1）训练样本的信息值

（2）第一棵树，属性，各叶节点的信息值

（3）第一棵树，属性，分割后导致的信息增益

（4）依次，计算每棵树导致的信息增益

（5）选择获得最大信息增益的属性进行划分

（6）以此类推，递归，继续划分

（7）当所有叶节点都是纯的，划分过程终止

计算过程

（1）训练样本的信息值(基于类的比例）

训练样本（用来创建树的数据集）在包含9个yes和5个no的根节点上，对应信息值

info([9,5])=0.940位 →总的信息

$$info([9,5]) = -\frac{9}{14}\log\frac{9}{14} - \frac{5}{14}\log\frac{5}{14}$$

$$= (-9\log\frac{9}{14} - 5\log\frac{5}{14})/14$$

$$= [-9(\log 9 - \log 14) - 5(\log 5 - \log 14)]/14$$

$$= [-9\log 9 - 5\log 5 + 14\log 14]/14$$

$$= [-9\times 3.169925 - 5\times 2.321928 + 14\times 3.807355]/14$$

$$= [-28.5293 - 11.6096 + 53.30297]/14$$

$$= 13.164/14 = 0.940286$$

Outlook	Temperature	Humidity	Windy	Play?
sunny	hot	high	false	No
sunny	hot	high	true	No
overcast	hot	high	false	Yes
rain	mild	high	false	Yes
rain	cool	normal	false	Yes
rain	cool	normal	true	No
overcast	cool	normal	true	Yes
sunny	mild	high	false	No
sunny	cool	normal	false	Yes
rain	mild	normal	false	Yes
sunny	mild	normal	true	Yes
overcast	mild	high	true	Yes
overcast	hot	normal	false	Yes
rain	mild	high	true	No

计算过程

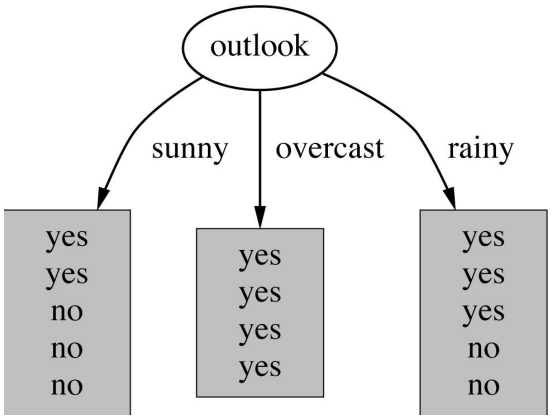
（2） 第一棵树，属性，各叶节点的信息值

基于天气(outlook)的划分，在叶节点的yes和no类的个数分别是[2,3]，[4,0]，和[3,2]，而这些节点的信息值分别是

info([2,3])=0.971位 →sunny

info([4,0])=0. 0位 →overcast

info([3,2])=0.971位 →rain



	YES	No	合计
sunny	2	3	5
overcast	4	0	4
rain	3	2	5
合计	9	5	

计算过程

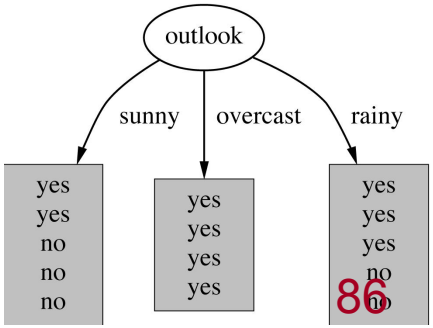
（3）第一棵树，属性，导致的信息增益

- 计算平均信息值。
- 根据天气的树导致的**信息增益**为：**基于类比例原来信息熵-基于天气属性划分之后得到的（平均）信息熵**

gain(outlook)=info([9,5])-
info([2,3],[4,0],[3,2])
=0.940-0.693=0.247位

$$Entropy(D)=\sum_{i=1}^{c}-P_i\log_2(P_i)$$

$$info([2,3],[4,0],[3,2])$$
$$=\frac{5}{14}info[2,3]+\frac{4}{14}info[4,0]+\frac{5}{14}info[3,2]$$
$$=\frac{5}{14}\left[-\frac{2}{5}\log\left(\frac{2}{5}\right)-\frac{3}{5}\log\left(\frac{3}{5}\right)\right]+\frac{4}{14}\left[-\frac{4}{4}\log\left(\frac{4}{4}\right)-0\log 0\right]+\frac{5}{14}\left[-\frac{3}{5}\log\left(\frac{3}{5}\right)-\frac{2}{5}\log\left(\frac{2}{5}\right)\right]$$
$$=\frac{5}{14}\times 0.971+\frac{4}{14}\times 0+\frac{5}{14}\times 0.971$$
$$=0.693\ 位$$



计算过程

（4）依次，计算每棵树导致的信息增益

为每个属性计算**信息增益**

gain(outlook)=0.247位

gain(temperature)=0.029位

gain(humidity)=0.152位

gain(windy)=0.048位

$$Gain(D,A) = Entropy(D) - \sum_{v \in Values(A)} \left(\frac{|D_v|}{|D|} Entropy(D_v) \right)$$

✓Values(A)是属性A所有可能值的集合。
✓D_v是D中属性A的值为v的子集，即$D_v=\{d|d \in D, A(d)=v\}$。
✓Entropy(D)是D未用属性A分割之前的熵。
✓Entropy(D_v)是D用属性A分割之后D_v的熵。

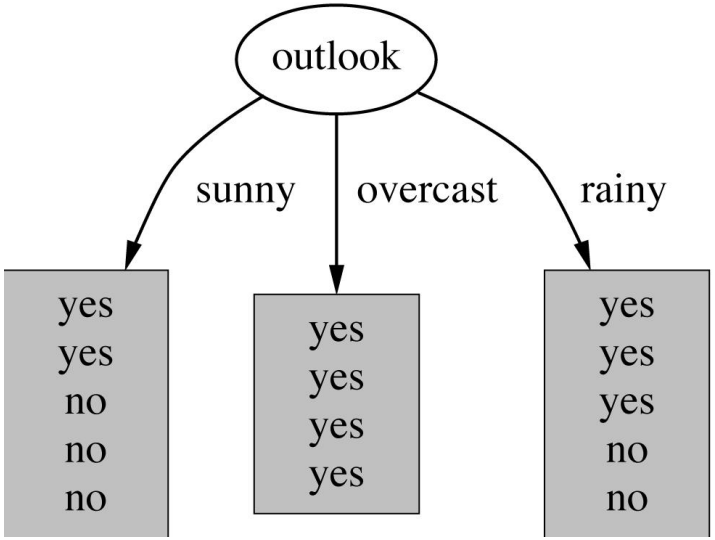
计算过程

（5）选择获得最大信息增益的属性进行划分

最大信息增益：gain(outlook)=0.247位

选择天气作为树的根节点的划分属性，其中一个子女节点是最纯的，并且这使它明显优于其他属性。

湿度是次佳选择，它产生了一个几乎完全纯的较大子女节点。



计算过程

（6）以此类推，递归，继续划分

递归继续选择

当天气为晴时，所达到的节点上的可能的深一层的分支

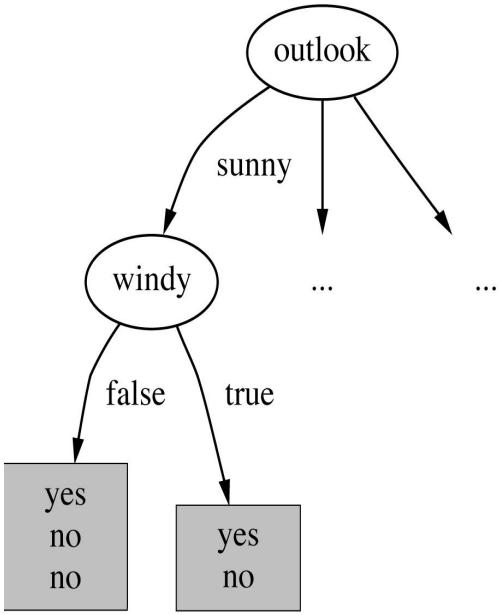
除天气外，其他属性产生的信息增益分别为：

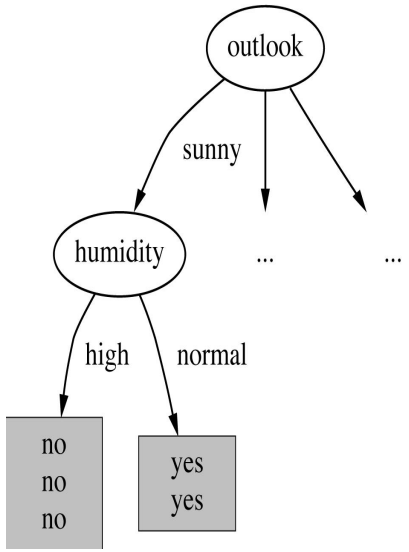
gain(temperature)=0.571位

gain(humidity)=0.971位

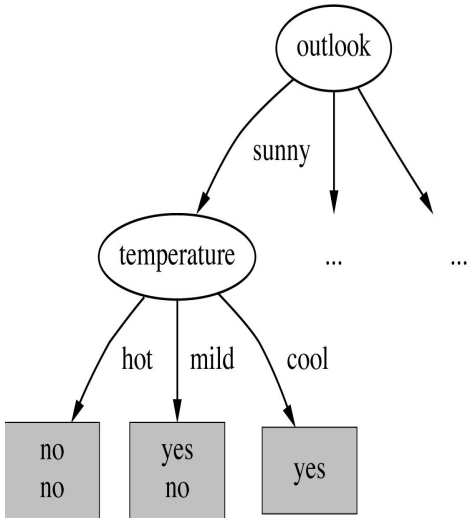
gain(windy)=0.020位

继续再选择湿度(humidity)作为划分属性

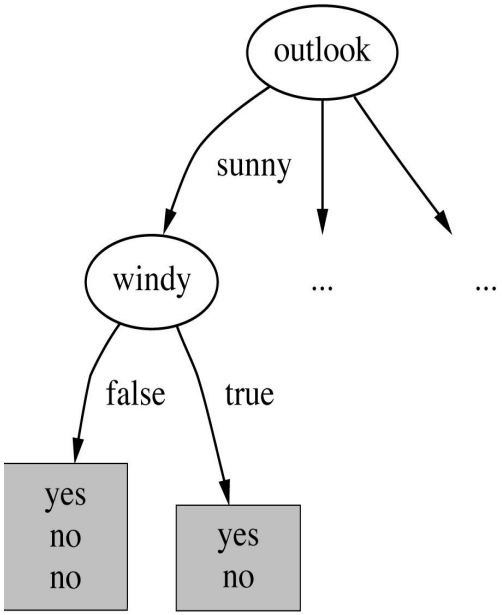




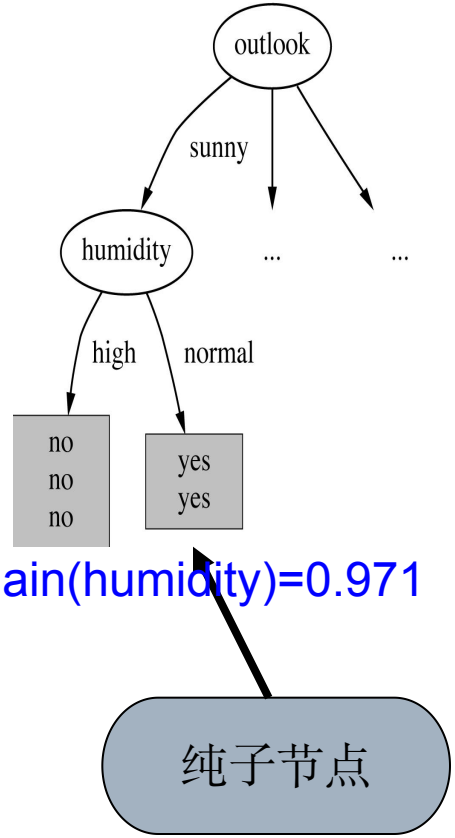
计算过程



gain(temperature)=0.571



gain(windy)=0.020



计算过程

（6）以此类推，递归，继续划分

天气，晴分支，气温，gain(temperature)=0.571位

天气，晴分支，湿度，gain(humidity)=0.971位

(纯的子女节点）

天气，晴分支，有风，gain(windy)=0.020位

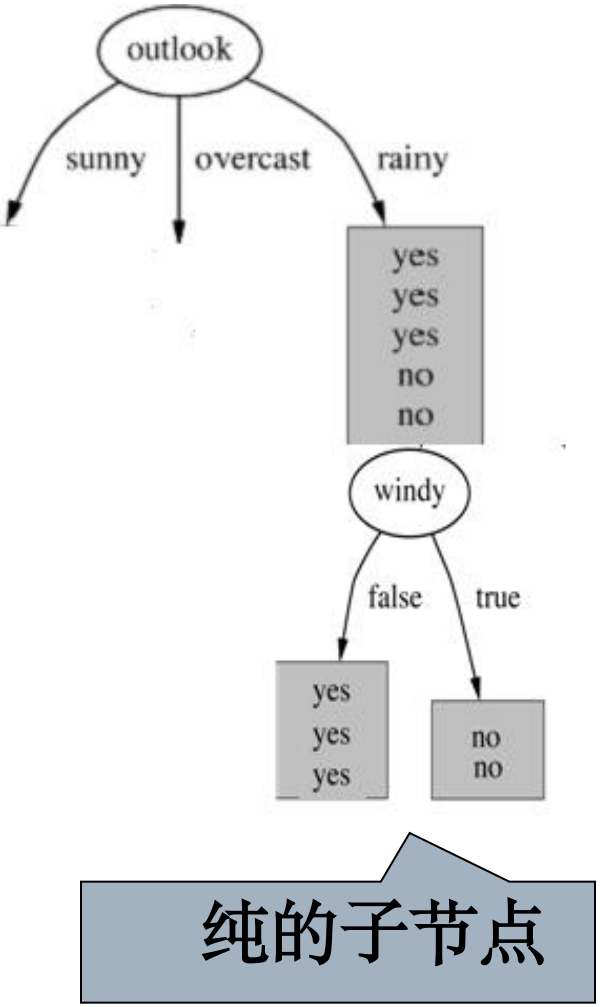
天气，雨分支，气温，gain(temperature)=0.020位

天气，雨分支，湿度，gain(humidity)=0.020位

天气，雨分支，有风，gain(windy)=0.971位

（纯的子女节点）

计算过程


outlook
sunny
overcast
rainy
yes
yes
yes
no
no
windy
false
true
yes
yes
yes
no
no
纯的子节点

计算过程

（7）**当所有叶节点都是纯的，划分过程终止**

理想情况下，当所有叶节点都是纯的而使过程终止时，即当它们包含的实例都具有相同类时该过程终止。

可能无法达到这种结果，因为无法避免训练集包含两个具有相同属性集，但具有不同类的实例。

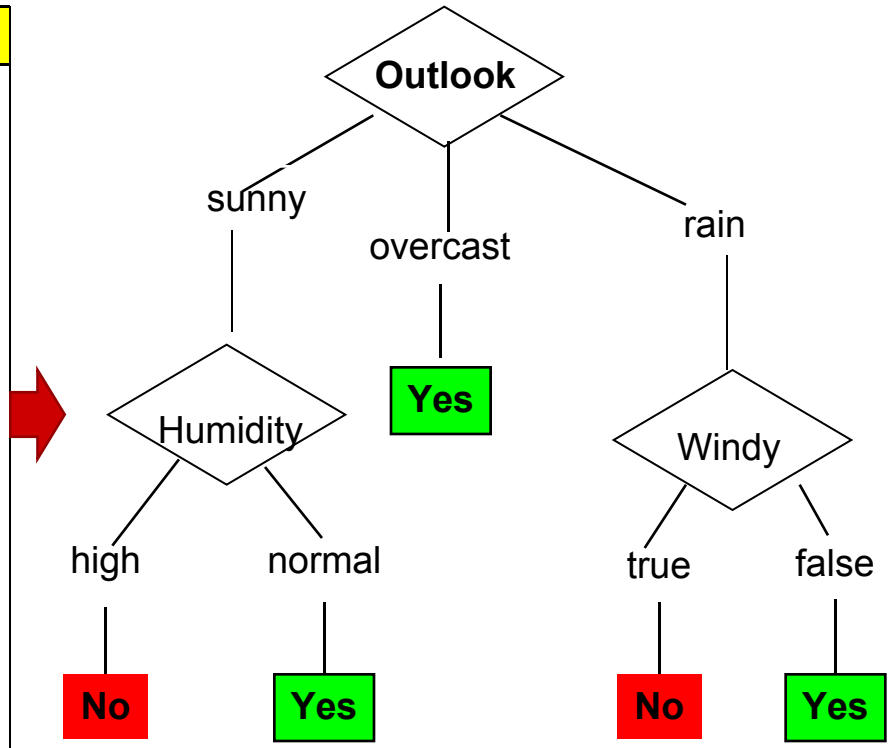
当数据不能进一步划分时，停止划分过程。

计算过程

- 基于Weather数据集

Outlook	Temperature	Humidity	Windy	Play?
sunny	hot	high	false	No
sunny	hot	high	true	No
overcast	hot	high	false	Yes
rain	mild	high	false	Yes
rain	cool	normal	false	Yes
rain	cool	normal	true	No
overcast	cool	normal	true	Yes
sunny	mild	high	false	No
sunny	cool	normal	false	Yes
rain	mild	normal	false	Yes
sunny	mild	normal	true	Yes
overcast	mild	high	true	Yes
overcast	hot	normal	false	Yes
rain	mild	high	true	No

最终构建好的决策树



1.4 决策树特点

✅ **优点：**

- **可解释性强**：规则清晰，易于人类理解（“白盒模型”）。
- **无需特征缩放**：对特征的量纲不敏感（不像 SVM、KNN）。
- **自动处理特征交互**：能捕捉非线性关系和特征组合。
- **支持数值型和类别型特征**（部分实现需预处理）。
- **训练和预测速度快**（尤其浅层树）。

❌ **缺点：**

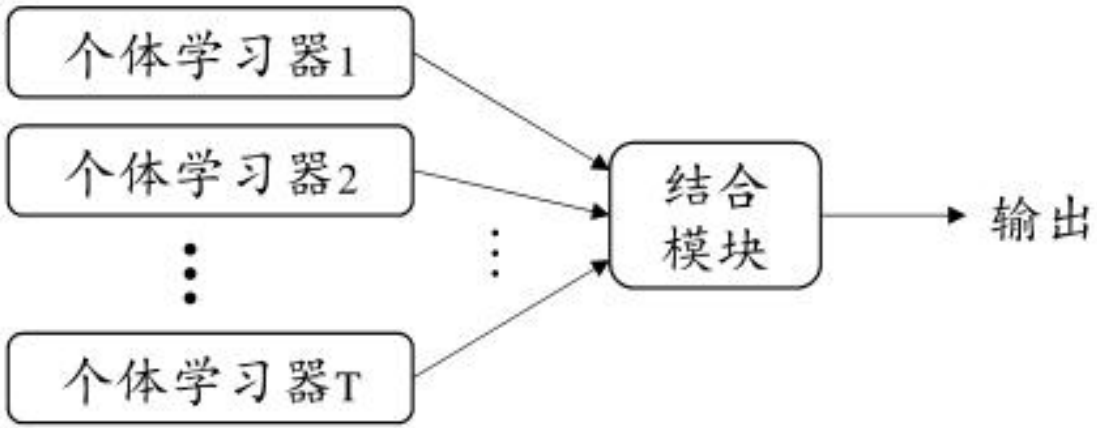
- **容易过拟合**：尤其当树很深、未剪枝时。
- **对数据扰动敏感**：训练集微小变化可能导致树结构大变。
- **难以表达异或（XOR）等复杂边界**（需很深的树）。
- **有偏性**：倾向于选择取值多的特征（ID3 问题）。

> 因此，**集成方法**（如随机森林、GBDT）常用来克服这些缺点。

1.4 集成学习

个体与集成

集成学习(ensemble learning)通过构建并结合多个学习器来提升性能



核心思想：三个臭皮匠赛过诸葛亮

1.4 集成学习

考虑一个简单的例子，在二分类问题中，假定3个分类器在三个样本中的表现如下图所示，其中 √ 表示分类正确，×表示分类错误，集成的结果通过投票产生。

	测试例1	测试例2	测试例3
h_1	√	√	×
h_2	×	√	√
h_3	√	×	√
集群	√	√	√

(a) 集群提升性能

	测试例1	测试例2	测试例3
h_1	√	√	×
h_2	√	√	×
h_3	√	√	×
集群	√	√	×

(b) 集群不起作用

	测试例1	测试例2	测试例3
h_1	√	×	×
h_2	×	√	×
h_3	×	×	√
集群	×	×	×

(c) 集群起负作用

集成个体应：好而不同

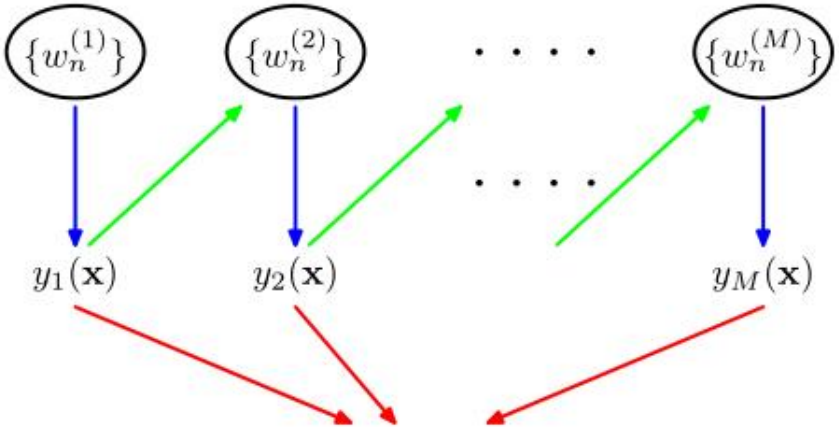
1.4 集成学习

- Boosting
- Bagging
- Bagging的扩展： 随机森林
- 结合策略
  - 平均法
  - 投票法
  - 学习法

1.4 集成学习

Boosting

- 个体学习器存在强依赖关系：每个弱学习器不是独立构建的，而是依赖于前一个学习器的输出和错误。即 每个弱学习器的目标是弥补之前模型的不足，共同提高整体模型的性能。
- 串行生成
- 每次调整训练数据的样本分布：每一轮训练后增加被前一轮模型错误分类的样本的权重，从而使后续的弱学习器更关注这些难分类的样本。



$$Y_M(\mathbf{x}) = \text{sign}\left(\sum_m^M \alpha_m y_m(\mathbf{x})\right)$$

1.4 集成学习

Bagging

- 个体学习器不存在强依赖关系
- 并行化生成
- 自助采样法

输入： 训练集 $D = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_m, y_m)\}$;
基学习算法 $\mathfrak{L}$;
训练轮数 T.

过程：

1: **for** $t = 1, 2, \ldots, T$ **do**
2: $\quad h_t = \mathfrak{L}(D, \mathcal{D}_{bs})$
3: **end for**

输出： $H(\boldsymbol{x}) = \arg\max_{y \in \mathcal{Y}} \sum_{t=1}^{T} \mathbb{I}(h_t(\boldsymbol{x}) = y)$

1.4 随机森林

- 随机森林 (Random Forest，简称 RF) 是 bagging 的一个扩展变种
- 数据采样的随机性
- 特征/属性选择的随机性

随机森林（Random Forest）是在 Bagging 的基础上再引入了随机特征，进一步提高每个基模型之间的独立性。在随机森林中，每个基模型都是一棵决策树。

1.4 随机森林

随机森林案例：判断蘑菇有没有毒

假设有一堆蘑菇，每只蘑菇有**颜色、形状、气味**等特征，要判断它有没有毒。

- **单棵决策树：**可能因为某次训练数据中“红色=有毒”太多，就死记硬背“所有红色蘑菇都有毒”——但现实中可能有例外，导致过拟合。

- **随机森林：**

  随机选数据：从所有蘑菇中有放回地随机抽样（比如抽1000次，有些蘑菇被抽多次，有些没被抽到）。

  随机选特征：每棵树在判断时，只看一部分特征（比如有的树只看颜色+气味，有的看形状+大小），避免所有树都依赖同一个强特征。

  每棵树独立判断：每棵树都用自己的数据和特征训练，给出“有毒”或“无毒”。

即使个别树犯错，整体结果依然稳健!

1.4 随机森林

- 随机森林的三大“随机”之处

1.数据随机：每棵树用不同的训练子集。

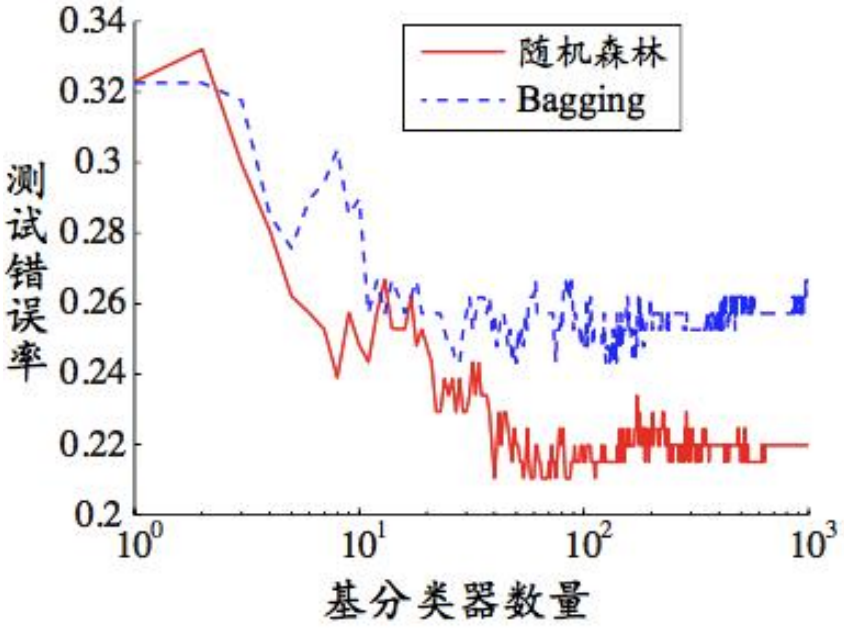
2.特征随机：每棵树分裂节点时，只从随机子集中选最佳特征（比如总共有20个特征，每次只看5个）。

3.结构随机：由于数据和特征都随机，每棵树长得都不一样。

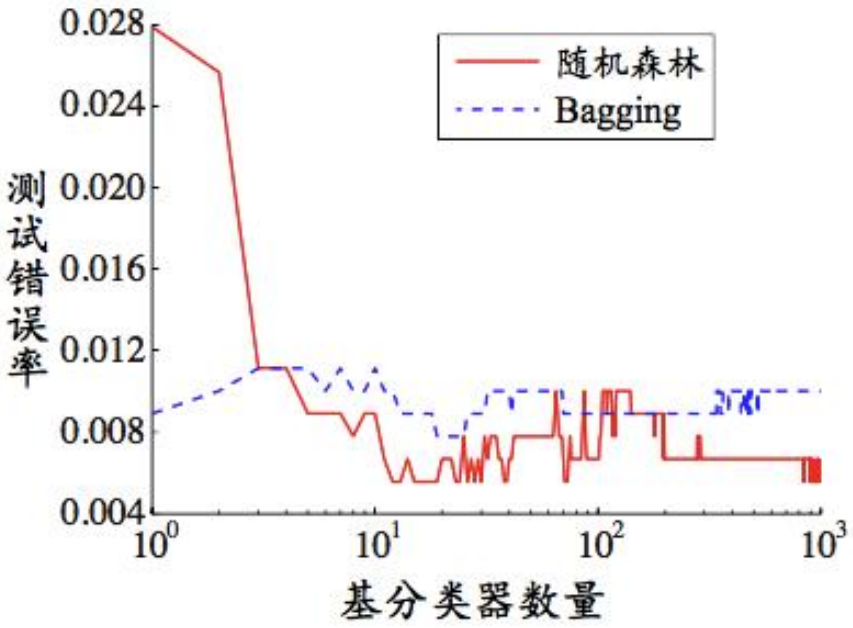
正是这种“多样性”，让随机森林不容易过拟合，泛化能力很强。

1.4 随机森林

□ Bagging与随机森林的对比



(a) glass 数据集



(b) auto-mpg 数据集

1.4 随机森林

- **随机森林** Random Forest 拓展模型



步骤1：从训练集中随机选择K个数据点

步骤2：构建与所选数据点（子集）关联的决策树。

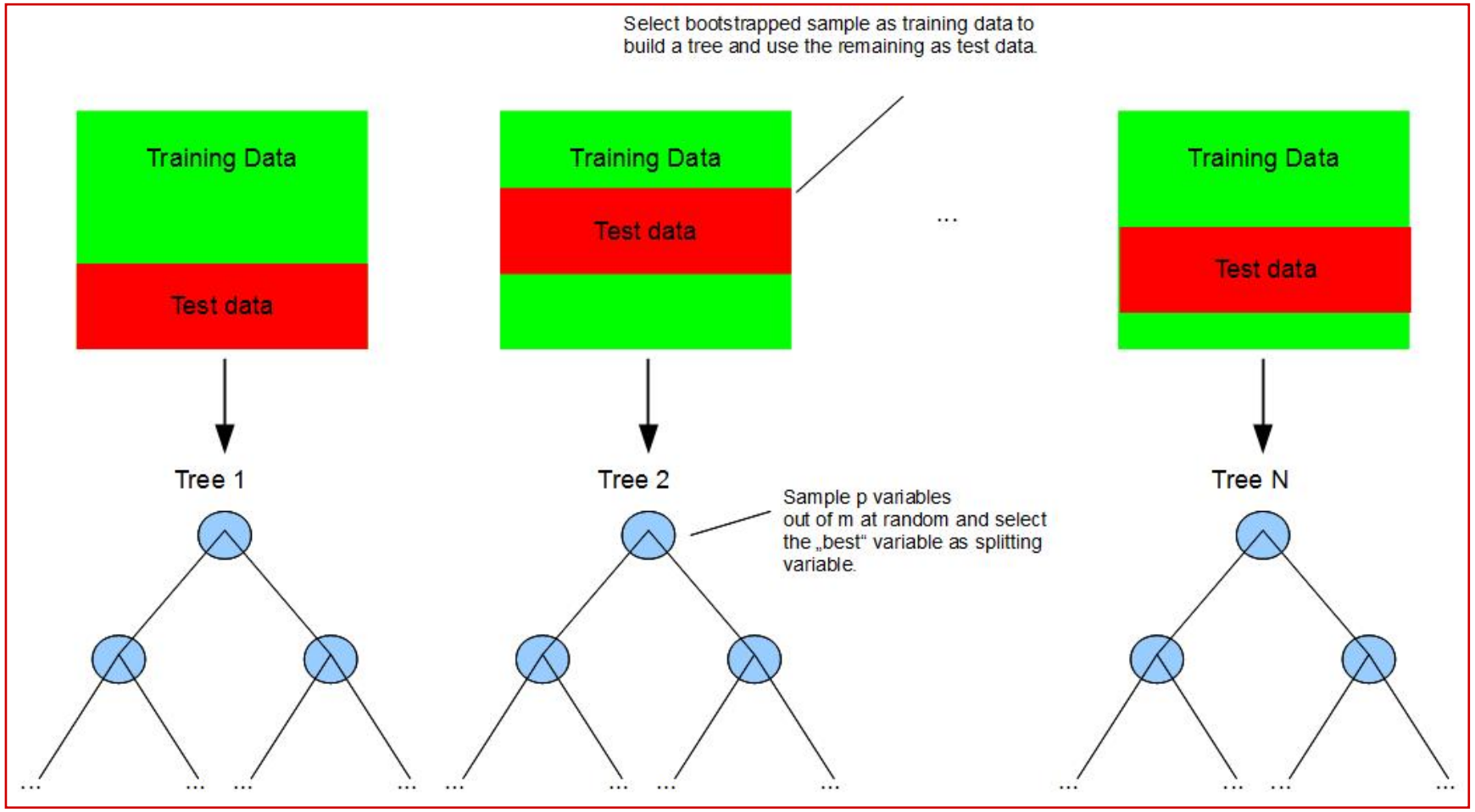
步骤3：为要构建的决策树选择数字N。

步骤4：重复步骤1和2，构建N个决策树

步骤5：对于新数据点，找到每个决策树的预测，并将新数据点分配给赢得多数票的类别。

1.4 随机森林

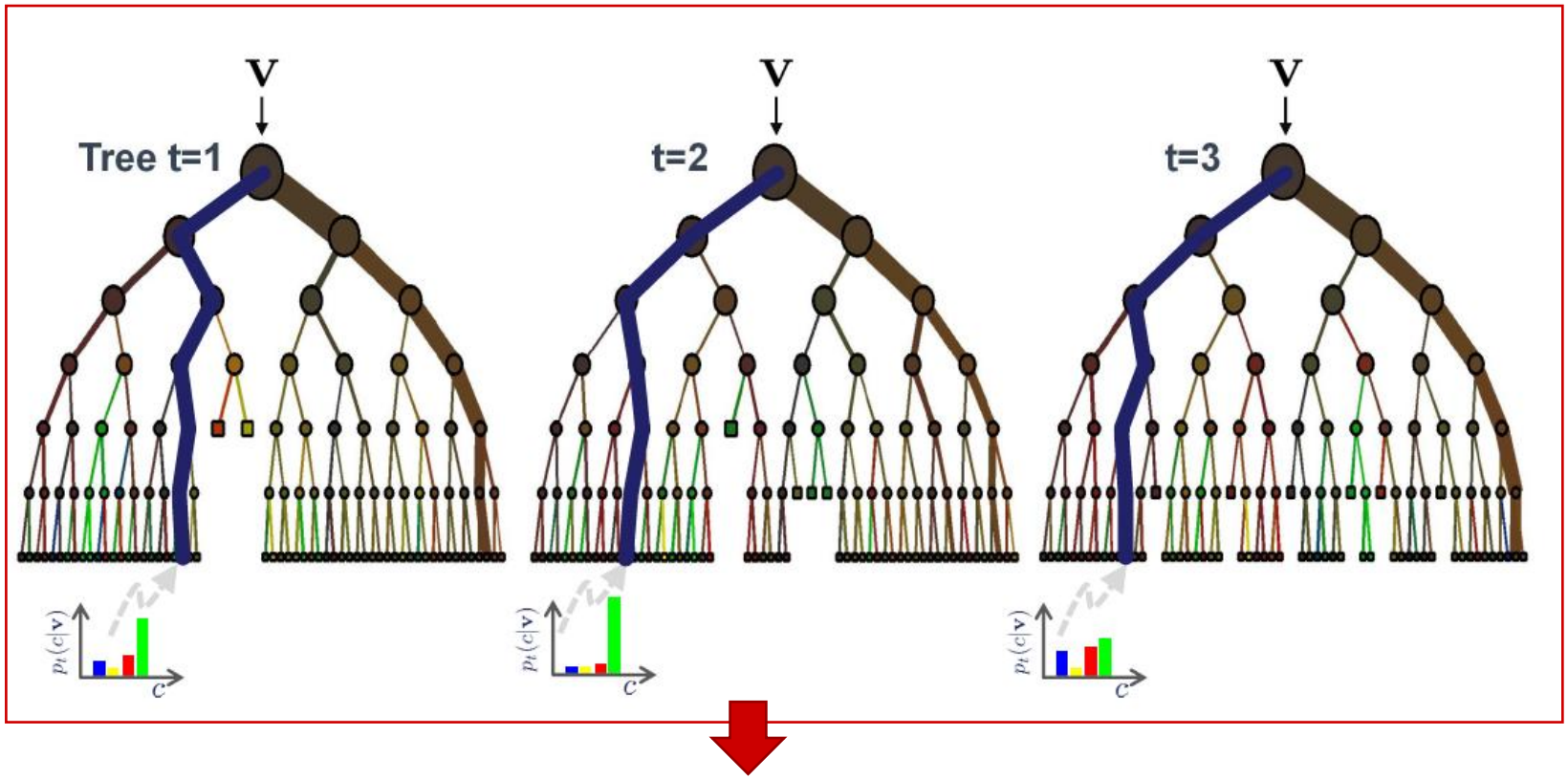
- **随机森林** Random Forest 训练-分类过程



训练阶段

1.4 随机森林

- **随机森林** Random Forest 训练-分类过程



分类阶段
投票集成

1.4 集成学习

结合策略 – 平均法

- 简单平均法

$$H(\boldsymbol{x}) = \frac{1}{T}\sum_{i=1}^{T} h_i(\boldsymbol{x}).$$

- 加权平均法

$$H(\boldsymbol{x}) = \sum_{i=1}^{T} w_i h_i(\boldsymbol{x}), \quad w_i \geq 0 \ \text{and} \ \sum_{i=1}^{T} w_i = 1.$$

加权平均法未必一定优于简单平均法?

1.4 集成学习

结合策略 – 投票法

- 绝对多数投票法（majority voting）

$$H(\boldsymbol{x}) = \begin{cases} c_j & \text{if } \sum_{i=1}^{T} h_i^j(\boldsymbol{x}) > \frac{1}{2} \sum_{k=1}^{l} \sum_{i=1}^{T} h_i^k(\boldsymbol{x}) \\ \text{rejection} & \text{otherwise .} \end{cases}$$

要求候选人或方案必须获得超过所有选票总数的一半（通常指超过50%的票数），才能被视为胜出。

- 相对多数投票法（plurality voting）

$$H(\boldsymbol{x}) = c_{\arg\max_j \sum_{i=1}^{T} h_i^j(\boldsymbol{x})}$$

在所有候选人中，获得票数最多的候选人胜出，无论该票数是否超过所有票数的一半。

- 加权投票法（weighted voting）

$$H(\boldsymbol{x}) = c_{\arg\max_j \sum_{i=1}^{T} w_i h_i^j(\boldsymbol{x})}$$